

ORIGINAL ARTICLE

Physics-informed neural network with symbolic regression for deriving analytical approximate solutions to nonlinear partial differential equations

Joy Das¹ · Bivas Bhaumik² · Soumen De¹ · Satyasan Changdar³

Received: 10 April 2025 / Accepted: 16 June 2025 / Published online: 6 July 2025

© The Author(s) 2025

Abstract

This work explores an interpretable approach based on physics-informed neural networks (PINNs) combined with symbolic regression (SR) to determine mathematical expressions for the predicted solutions of nonlinear partial differential equations that describe arterial blood flow influenced by external magnetic fields. PINNs are excellent at capturing the underlying physics, but can be computationally intensive and hard to interpret. The predictive power of PINN has been combined with evolutionary symbolic regression using PySR, an open-source Python module that employs genetic programming and customizable mathematical operators, including trigonometric, exponential, and arithmetic functions. This hybrid approach enables the derivation of concise, transparent mathematical expressions that closely replicate the behavior of these complex systems. This blend of PINNs and symbolic regression helps us to better understand how pulsatile blood flow and magnetic fields interact in the viscoelastic arterial circulation. The comparison graphs of the SR model and the PINN-predicted data at different time scales signify a better fit for the discovered mathematical expressions with data. As illustrated by the low mean squared error and statistical validation on residual losses, the symbolic expressions are extremely accurate and quick enough for real-time execution. Additionally, the solutions provided by the PINN are validated numerically to demonstrate the effectiveness of the proposed method. Our results demonstrate that combining symbolic regression with PINNs provides practical and interpretable solutions in biofluid mechanics, offering a more transparent and reliable alternative to traditional methods.

Keywords Physics-informed neural network (PINN) · Symbolic regression (SR) · Interpretable machine learning · Nonlinear partial differential equations (PDEs) · Magnetic biofluid dynamics

1 Introduction

Arterial blood flow in the cardiovascular system is a fundamental aspect of the human circulatory system, as it is responsible for transporting oxygenated blood from the heart to various tissues and organs of the body [1]. The heart circulates blood through a complex network of arterial tubes. Unlike inert tubes, the arteries react to changing flow and pressure circumstances by expanding or contracting to meet changing hemodynamic demands [2]. Again, blood is considered a biomagnetic fluid, as it contains hemoglobin molecules, which are generally high concentrations of iron oxides present in red blood cells [3]. Therefore, when a magnetic field is applied, the flowing nature of blood changes due to its magnetic properties [4]. One-dimensional (1D) models are used to evaluate the effects of blood pressure and flow rate on the arterial and venous walls in the biological circulation



system [5]. The nonlinear wave dynamics of fluid flow through viscoelastic tubes necessitate particular attention, as these tubes serve as analogs to major arteries in the human circulatory system. Understanding this dynamic behavior is critical for comprehending the intricacies of fluid dynamics in such systems [5]. Several cardiovascular diseases and hypertension occur in humans due to the improper flow of blood in the arteries. One-dimensional (1D) fluid dynamical models predict flow, area, and pressure without being nonlinear. The blood flow dynamical system has various damping components, including fluid viscosity, wall viscoelasticity, and vessel geometry variations [5, 6]. Thus, capturing pressure pulses and flow rates of blood under the influence of a magnetic field, as well as characterizing the bending behavior of the vessel wall in arteries, is crucial to overcome various cardiovascular problems. Analyzing this biomagnetic fluid is beneficial for the development of magnetic devices for the treatment of malignant tumors, cell separation, and transporting drugs using magnetic particles [3, 7]. Therefore, a mathematical model should account for mechanical interactions between blood, artery walls, and tissues.

After advancing in computer technologies, deep learning algorithms have become more interesting and widely used tools in many branches of applied mathematics and scientific problems [8]. Artificial neural network (ANN) is a deep learning algorithm inspired by the human nervous system and is used for artificial intelligence and machine learning [9]. This neural network is constructed with one input layer, one output layer, and several hidden layers with various activation functions in the hidden layers. This method effectively solves various complex mathematical problems with high efficiency. Before using the ANN for problem-solving, the proposed neural network must be trained to learn the underlying patterns and relationships within the data by adjusting its internal parameters, such as weights and biases, through an optimization technique [9]. New computational procedures work as a motivation to find different solutions to various mathematical equations. Some numerical methods are used to solve partial differential equations (PDEs) through a discretization process. In recent years, deep learning has become very popular among researchers to solve PDEs [10]. Physics-informed neural network (PINN) is an emerging machine learning algorithm that effectively solves higher-order nonlinear PDEs with great efficiency. PINNs incorporate the underlying physical laws directly into the training process of the neural network, and this is achieved by embedding the PDEs into the loss function as additional regularization terms [10, 11]. As a result, the network learns from the data perfectly and is also guided by the underlying physics, which improves generalization and assures that the learnt solution is consistent with the known physical behavior of the system. The capacity of PINN to solve PDE problems offers advantages such as continuous and differentiable approximate solutions and reduced memory requirements. PINN has the benefit of utilizing automated differentiation methods through the Tensorflow and PyTorch libraries [10, 12]. This approach improves PINN performance by estimating the initial and boundary conditions of the PDEs.

PDEs are essential for modeling intricate systems in various scientific and engineering disciplines, extending beyond cardiovascular flows. Addressing these nonlinear and frequently high-dimensional PDEs presents significant challenges, which have led to an increased use of neural network-based approaches. The significance of linear and nonlinear PDEs extends across many disciplines, including solid mechanics, fluid dynamics, nonlinear optics, plasma physics, and the fuzziness concept in pattern classification [13–15]. These equations encapsulate complex physical phenomena such as turbulence, dispersion, and reaction–diffusion processes. Moreover, PINNs have been successfully applied in diverse areas such as natural language processing, computer vision, stock market modeling, and several other domains [16, 17]. PINNs offer benefits such as solutions that are free from mesh constraints, continuous differentiability, and the straightforward incorporation of physical laws, making them effective tools for addressing PDEs in multiple contexts [10, 11]. These advancements highlight the broad applicability of partial differential equations (PDEs) and underscore the growing importance of machine learning-based analytical approaches in discovering interpretable solutions throughout scientific disciplines.

Nowadays, PINNs have emerged as the leading machine learning approach to predict the solutions of high-dimensional PDEs accurately. PINNs are typically viewed as black-box solvers due to their difficulties in interpreting neural solutions, which is a significant restriction [18, 19]. The interpretability of a machine learning model is determined by its ability to trace the logical or mathematical linkages between input and output.

Furthermore, optimal machine learning models are interpretable when expressed as mathematical formulations. Symbolic regression (SR) is a fast-growing discipline of machine learning that uses trained datasets to create symbolic mathematical formulae [20, 21]. The learning process for the SR model is complex as it requires several mathematical operations and constant values to represent data effectively. Genetic programming approaches are commonly employed to accomplish this [20]. Limited research has been conducted on interpreting physics-informed neural networks using a symbolic regression approach [21]. Deep neural networks, or PINNs, can solve PDEs in natural sciences like physics. This enables SR models to obtain mathematical formulas for approximate analytical solutions. SR models may be constructed directly from projected solutions using physics-informed neural networks, changing “black-box” models into more interpretable “white box” models. The mathematical expressions of the solutions of the PDEs can be impactful in identifying the flowing behavior as well as characteristics of the motion of the blood in human arteries and making suitable medical devices accordingly to diagnose several cardiovascular diseases.

The main aspect of this work is to create a hybrid framework that combines machine learning with genetic programming to use symbolic regression in predicting the analytical solution of the fifth-order evolutionary equation and forced Burgers’ equation, which is derived from a one-dimensional (1D) arterial blood flow model by using the reductive perturbation technique. The use of symbolic regression in finding an approximate analytical form of the solution of a fifth-order partial differential equation is present in [21]. In this aspect, we have explored the detailed mechanism of how symbolic regression uses genetic programming to effectively identify the mathematical forms of the solutions, combined with interpretable data of the PINN model. The method may be used to look at the important aspects of the nonlinear elasticity and viscoelastic characteristics of tube walls, as well as their bending nature. Our analysis uses the fifth-order nonlinear evolution equation, which is an extension of the Kawahara equation, to analyze the bending dynamics of the viscoelastic tube wall. This study examines the use of PINNs and the symbolic regression technique to solve a nonlinear evolutionary equation with fifth-order dispersion under pressure waves. Moreover, validation entails comparing expected solutions to residuals of the partial differential equation to conclude the full inquiry.

2 Fundamental formulations describing arterial blood flow

Arterial blood circulation is frequently represented through partial differential equations that illustrate the dynamic interactions between blood, regarded as a non-Newtonian, biomagnetic fluid, and the flexible arterial walls [22]. These equations are formulated from the conservation laws pertaining to mass and momentum, and they have been further enhanced to reflect the impacts of viscoelasticity and external forces like magnetic fields. The resulting system often includes nonlinear and higher-order dispersive terms that describe wave propagation, damping effects, and elasticity of the arterial walls.

To better represent the incompressible flow of fluid in an axially symmetric viscoelastic artery, the continuity and axial momentum equations are formulated as follows:

2.1 Equation of continuity

For incompressible fluids like blood [23], the continuity equation conveys the principle of mass conservation.

$$\frac{\partial \bar{v}_x}{\partial x} + \frac{1}{r} \frac{\partial (r \bar{v}_r)}{\partial r} = 0 \quad (1)$$

2.2 Equation of momentum

The axial component of Navier's Stokes equations determines the fluid flow properties within an axially symmetric artery tube [5, 24]. Here, the Lorentz force $\mathbf{J} \times \mathbf{B}$ works as an external body force in the momentum equation due to the presence of a small magnetic field effect ($\mathbf{B}(B_0, 0, 0)$), normal to the blood flow. The current density $\mathbf{J}$ is given by Ohm's law under the assumption of negligible electric field and low magnetic Reynolds number and is given by

$$\mathbf{J} = \sigma(\mathbf{v} \times \mathbf{B}) \quad (2)$$

Here, σ is the electrical conductivity of the fluid and $\mathbf{v}$ represents the fluid velocity vector.

2.2.1 Axial component

$$\frac{\partial \bar{v}_x}{\partial t} + \bar{v}_r \frac{\partial \bar{v}_x}{\partial r} + \bar{v}_x \frac{\partial \bar{v}_x}{\partial x} + \frac{1}{\rho} \frac{\partial \bar{P}}{\partial x} = \mu^0 \left[\frac{\partial^2 \bar{v}_x}{\partial r^2} + \frac{1}{r} \frac{\partial \bar{v}_x}{\partial r} + \frac{\partial^2 \bar{v}_x}{\partial x^2} \right] - \sigma B_0^2 \bar{v}_x \quad (3)$$

Here, the components of the flow velocity in the axial and radial directions are denoted by $\bar{v}_x$ and $\bar{v}_r$, respectively. The fluid's kinematic viscosity is represented by μ^0 , and its density by ρ . The pressure field within the fluid, $\bar{P} = \bar{P}(x, t)$, is considered to vary with both spatial position and time.

We now use the following non-dimensional variables in the equations above to reduce them to one-dimensional form:

$$r = R'_0 r', \quad x = \lambda' x', \quad t = \frac{\lambda'}{V'_0} t', \quad \bar{v}_x = V'_0 v'_x, \quad \bar{v}_r = U'_0 v'_r, \quad \bar{P} = \rho V_0'^2 P'. \quad (4)$$

Also note that

$$\tilde{\omega} = \frac{R'_0}{\lambda'} = \frac{U'_0}{V'_0} \quad (5)$$

where $\tilde{\omega}$ is a small parameter. Thus, using the above dimensionless variables and the condition $\frac{V'_0 R'_0}{U'_0 \lambda'}$ Eq. (1) takes the following form

$$\frac{\partial(r' v'_r)}{\partial r'} + \frac{\partial(r' v'_x)}{\partial x} = 0 \quad (6)$$

Now, using the above dimensionless variables and dividing Eq. (3) by $V_0'^2$, multiplying it by $r' \lambda'$, neglecting the term of order $\tilde{\omega}^2$, and thus using Eq. (5), we get the following form of the momentum equation:

$$\frac{\partial}{\partial t} (r' v'_x) + \frac{\partial}{\partial r'} (r' v'_r v'_x) + \frac{\partial}{\partial x} (r' v_x'^2) + \frac{\partial}{\partial x} (r' P') = \frac{v^0 \lambda}{V'_0 R_0'^2} \left[\frac{\partial}{\partial r'} \left(r' \frac{\partial v'_x}{\partial r'} \right) \right] - \frac{\lambda'_0}{V'_0} \sigma B_0'^2 r' v'_x \quad (7)$$

The power generalization velocity profile is expressed as follows [25, 26]:

$$v'_x(r', x, t) = \frac{\bar{\gamma} + 2}{\bar{\gamma}} \left[1 - \left(\frac{r'}{\bar{R}} \right)^\gamma \right] \bar{V}'(x, t), \quad \bar{V}'(x, t) = \frac{2}{\bar{R}^2} \int_0^{\bar{R}'(x, t)} v(r', x, t) r' dr' \quad (8)$$

and

$$\alpha = \frac{1}{\bar{R}^2 \bar{U}^2} \int_0^{\bar{R}'} 2 v_x'^2 r' dr' \quad (9)$$

Here, $\bar{R}' = \bar{R}'(x, t)$ denotes the instantaneous radius of the arterial tube, while $\bar{\gamma}$ characterizes the sharpness of the axial velocity profile. The parameter α serves as a correction factor, and the term $\bar{v}_x$ refers to how the axial velocity component varies along the radial direction, whereas the average velocity across the cross section is given by $\bar{V}'(x, t)$.

To derive the averaged quantities, we integrate both the continuity and momentum equations over the radial domain, specifically from $r' = 0$ to $r' = \bar{R}'$. In doing so, we apply the streamline boundary condition, which assumes that the fluid velocity remains tangential along the wall surface, as

$$[v_{r'}]_{r'=\bar{R}'} = \frac{\partial \bar{R}'}{\partial x} [v'_x]_{r'=\bar{R}'} + \frac{\partial \bar{R}'}{\partial t} \quad (10)$$

Now, integrating continuity Eq. (1) and using the streamline condition, we obtain the averaged continuity equation as

$$\frac{\partial}{\partial t} \bar{R}'^2 + \frac{\partial}{\partial x} (\bar{R}'^2 \bar{V}') = 0 \quad (11)$$

Similarly, integrating Eq. (3) and using Eqs. (6) and (7), and also utilizing the streamline condition, we get the averaged momentum equation as

$$\frac{\partial}{\partial t} (\bar{R}'^2 \bar{V}') + \frac{\partial}{\partial x} (\alpha \bar{R}'^2 \bar{V}'^2) + \bar{R}'^2 \frac{\partial P'}{\partial x} = \frac{2v^0 \lambda' \bar{R}'}{V_0' \bar{R}_0'^2} \left[\frac{\partial v'_x}{\partial r'} \right]_{r'=\bar{R}'} - \frac{\lambda' \sigma B_0^2}{2V_0'} \bar{V}' \bar{R}'^2 \quad (12)$$

Utilizing the velocity profile in Eq. (8) and introducing $A = \pi \bar{R}'^2$, the one-dimensional form of the averaged equations, which define the cross section, is expressed as

2.2.2 Equation of Fluid mass in one-dimensional form

$$\frac{\partial}{\partial t} \bar{R}'^2 + \frac{\partial}{\partial x} (\bar{R}'^2 \bar{V}') = 0 \quad (13)$$

2.2.3 Momentum Equation in one-dimensional form

$$\frac{\partial \bar{V}'}{\partial t} + \bar{V}' \frac{\partial \bar{V}'}{\partial x} + \frac{\partial P'}{\partial x} = -2(\bar{\gamma} + 2) \frac{v^0 \lambda' \bar{V}'}{V_0' \bar{R}_0'^2 A} - \frac{\lambda \sigma B_0^2}{2V_0'} \bar{V}' \quad (14)$$

In this case, the arterial tube's cross-sectional area is $A(x, t)$. When A is substituted, Eq. (13) becomes

$$\tilde{R}'_t + \tilde{V}'\tilde{R}'_x + 0.5\tilde{R}'\tilde{V}' = 0. \quad (15)$$

To account for miniature tube wall disruptions, the function $\tilde{\zeta}(x, t)$ is defined as follows:

$$\tilde{R}'(x, t) = \bar{R} + \tilde{\zeta}(x, t), \quad \bar{R} = \text{equilibrium radius} \quad \|\tilde{\zeta}\| < \bar{R}. \quad (16)$$

Therefore, Eq. (15) is simplified as follows:

$$\tilde{\zeta}_t + 0.5\bar{R}\tilde{V}'_x + 0.5\tilde{\zeta}\tilde{V}'_x + \tilde{V}'\tilde{\zeta}_x = 0. \quad (17)$$

To resolve the system comprising two equations with three unknowns, as indicated in Eqs. (13) and (14), it is necessary to establish an extra relationship that guides the motion of the tube wall with the viscoelastic property. In this analysis, we assume a uniform arterial wall where both the deformation of the wall and its thickness are negligible compared to the tube radius. The wall strain is considered to be a function of the tube radius, and the equilibrium radius is assumed to be significantly smaller than the wavelength of the propagating wave phenomena. Additionally, we assume that the fluid pressure remains uniform across the cross-sectional area of the tube, varying only as a function of spatial coordinates and time [27].

Now, we can determine the equation for radial motion in a viscoelastic tube. As stated by Giusti and Mainardi [28], the inner pressure, represented by $\bar{P}(x, t)$, and the wall radius, which is denoted by $\tilde{\zeta}(x, t)$ of the tube, satisfy the following relationship:

$$\bar{P} - P_0 = \rho h_0 \tilde{\zeta}_{tt} - k h_0 \tilde{\zeta}_{xx} - \chi h_0 \tilde{\zeta}_{txx} + K \tilde{\zeta}_{xxxx} + \mu \tilde{\zeta}_t + \frac{\kappa h_0}{\bar{R}} \tilde{\zeta} + \frac{\kappa_2 h_0}{\bar{R}^2} \tilde{\zeta}^2. \quad (18)$$

where

$$\kappa_2 \equiv \kappa_1 \bar{R} - 2\kappa,$$

h_0 is the wall thickness in its stable condition, P_0 is the constant pressure on the tube's outer surface, and ρ_v is the volume density of the tubes. While χ represents the viscosity coefficient of the tube material, μ is the proportionality coefficient that connects the resistance of the medium to the motion of the viscoelastic wall. Furthermore, K represents the proportionality coefficient, and the linear and nonlinear elasticity coefficients of the tube are denoted by κ and κ_1 , respectively. For large Reynolds numbers, we focus on studying nonlinear waves with deep learning approaches utilizing the long-wave approximation. With these assumptions, Eq. (14) simplifies to:

$$\tilde{V}'_t + \tilde{V}'\tilde{V}'_x + \tilde{P}'_x = -\frac{\lambda \sigma B_0^2}{2V'_0} \tilde{V}'. \quad (19)$$

As a result, Eqs. (17), (18), and (19) together depict a fluid flow model in one dimension (1D) inside a viscoelastic tube. Additionally, these equations depict the blood flow system in the arteries of the human body.

Using the dimensionless variables in Eqs. (17), (18), and (19) along with the dimensionless variable

$$\tilde{\zeta} = \frac{\bar{R}}{2} \zeta', \quad V_0 = \sqrt{\frac{\kappa h_0}{2\rho}}$$

The following form (without the primes) represents the system of flow equations in dimensionless variables:

2.3 Non-dimensional flow equations

$$\begin{aligned}\zeta_t + \tilde{V}_x + \frac{1}{2}\zeta\tilde{V}_x + \tilde{V}\zeta_x &= 0 \\ \tilde{V}_t + \tilde{V}\tilde{V}_x + P_x &= M\tilde{V} \\ P &= \gamma\zeta_{tt} - \beta\zeta_{xx} + \lambda\zeta_t - \delta\zeta_{txx} + v_1\zeta_{xxxx} + \alpha\zeta + \alpha_1\zeta^2 + \alpha_2P_0.\end{aligned}\quad (20)$$

$$\begin{aligned}\text{Where, } \alpha &= \frac{\kappa h_0}{2\rho V_0^2} = 1, \quad \beta = \frac{kh_0\tilde{R}}{2\lambda^2\rho V_0^2}, \quad \gamma = \frac{h_0\tilde{R}}{2\lambda^2} \\ \delta &= \frac{\chi h_0}{2\lambda^3\rho V_0}, \quad \lambda = \frac{\mu\tilde{R}}{2\lambda\rho V_0}, \quad \alpha_1 = \frac{\kappa_2 h_0}{4\rho V_0^2}, \quad v_1 = \frac{K\tilde{R}}{2\lambda^4\rho V_0^2}, \quad \alpha_2 = \frac{1}{\rho V_0^2}, \quad M = \frac{-\lambda\sigma B_0^2}{2V_0}\end{aligned}\quad (21)$$

Nonlinear evolutionary equations for describing the blood flow in a viscoelastic tube can be obtained by the use of the perturbation technique. The reductive perturbation approach [29, 30] may be used to build approximation solutions for system Eq. (20). Once the direction of wave propagation has been determined, it makes sense to use “slow” time variables to investigate the development of the perturbations since pressure waves (also known as pulse waves) have a significantly greater characteristic velocity as compared to flow velocities. The following variables can be used in the following step to determine the solution to the system of equations (20):

$$\begin{aligned}\zeta^* &= \omega^p \zeta', \quad v^* = \omega^p v', \quad P^* = 1 + \omega^p P', \quad M^* = \omega^{p+1} M' \quad p \in N \\ \bar{\xi} &= \omega^m (x - t), \quad \bar{t} = \omega^n t, \quad m, n \in Q, \quad n > m\end{aligned}\quad (22)$$

$$\frac{\partial}{\partial x} = \omega^m \frac{\partial}{\partial \bar{\xi}}, \quad \frac{\partial}{\partial t} = \omega^n \frac{\partial}{\partial \bar{t}} - \omega^m \frac{\partial}{\partial \bar{\xi}}\quad (23)$$

By applying the relations given in Eqs. (22) and (23), we can derive a corresponding system of PDEs (i.e., Eqs. (24) and (25)) by choosing appropriate values for the indices n , m , p , and q and comparing the powers of ω in the perturbation method. These choices help simplify the general form of the equations and lead to a more specific, solvable structure. A detailed and rigorous derivation of these governing equations can be found in the works of [21], where the authors systematically develop the theoretical framework. Additionally, the magnetic field influence (the parameter M) can be seamlessly incorporated into the system using the perturbation technique. This approach allows for the controlled introduction of M as a small parameter, enabling the analysis of its effects on the overall dynamics of the system without drastically altering the core structure of the equations.

$$v_t + vv_x = \frac{1}{2}v_{xx} + \frac{M}{2}v,\quad (24)$$

$$v_t + vv_x + \frac{1}{2}v_{xxx} + \frac{1}{2}v_{xxxx} = \frac{M}{2}v\quad (25)$$

3 An advanced deep learning framework designed to effectively address nonlinear wave equations

In this section, we introduce an advanced technique of machine learning algorithms, specifically deep neural networks, for solving nonlinear PDEs. Current research uses a variety of frameworks, including recurrent neural networks, physics-guided neural networks (PgNNs), and PINNs, to comprehend the physics in scientific computing [21, 31, 32]. However, we prefer to use deep neural networks to solve nonlinear evolutionary equations to study wave dynamics. For most PDEs, this method is more suitable and easier to train. A deep neural network is a basic feed-forward neural network and may be thought of mathematically as a composition of functions with a very high approximation capacity [33]. Generally referred to as the input layer, output layer, and different hidden layers, a network architecture is constructed with three separate levels in the field of deep learning.

Let us consider a function $\Phi_k(\mathbf{x}, \mathbf{t}) : \mathbb{R}^2 \rightarrow \mathbb{R}^{d_{\text{sol}}}$ to perfectly define the network architecture with inputs (x, t) . Here the neural network consists of $(k - 1)$ hidden layers, with N_i neurons present in each hidden layer ($1 \leq i \leq k, N_k = d_{\text{sol}}$), for making the connection between input and output layer as well as transmitting the outputs from previously hidden layer to next input hidden layer. As explained in [31], we disclose the results from every hidden layer based on the type of deep neural network model:

$$\Phi_i(\mathbf{x}, \mathbf{t}) = \sigma_{\text{act}}(\mathcal{W}_i^T \Phi_{i-1}(\mathbf{x}, \mathbf{t}) + \mathbf{b}^i) \in \mathbb{R}^{N_i} \quad \text{for } 1 \leq i \leq k - 1.$$

Here, weight matrix is represented by $\mathcal{W}_i^T \in \mathbb{R}^{k_i \times k_{i-1}}$, bias vector is symbolizes as $\mathbf{b}^i \in \mathbb{R}^{N_i}$, and σ_{act} represents the nonlinear activation function in each hidden layers. In deep learning aspects, a specific choice of activation function is very important for better predicting performance. Nonlinear activation functions such as σ_{act} are more capable of learning complicated functions with induced nonlinearity of the network. The final layer is then defined as follows, after we use a linear activation function in the output layer.

$$\Phi_k(\mathbf{x}, \mathbf{t}) = \mathcal{W}_k^T \Phi_{k-1}(\mathbf{x}) + \mathbf{b}^l \in \mathbb{R}^{d_{\text{out}}}.$$

Regression tasks that seek to predict continuous variables frequently use a linear activation function in the output layer. By convention, the model chooses the linear activation function if the activation function is not specified in the output layer.

3.1 Configuring the problem using the PINN approach

Let us consider a general form of Partial Differential Equations with independent variables x and t , indicated as:

$$\mathcal{F} \equiv v_t + \mathcal{G}_v(x, t; v_x, v_{xx}, v_{xxx}, v_{xxxx}, v_{xxxxx} \dots) = 0, \quad \mathbf{x} \in \mathcal{S}, \quad \mathbf{t} \in (0, \tilde{T}), \quad \text{where} \quad (26)$$

$\mathcal{S}$ symbolize the spatial domain and $(0, \tilde{T})$ is the temporal domain subject to a well-defined Dirichlet boundary condition:

$$\mathcal{B}(v; x, t) = 0 \quad \text{on } \partial\mathcal{S}. \quad (27)$$

In the above expression, $\mathcal{B}(v; x, t)$ denotes the specified boundary conditions (such as Dirichlet, Neumann, or Periodic), $\mathcal{G}_v$ portrays the function with induced nonlinearity, and v symbolizes the solution of the partial differential equations specified in this study. In this aspect, the two-dimensional rectangular domain is denoted by $\mathcal{S} \times (0, \tilde{T})$, and $\partial\mathcal{S}$ represents the boundary points in the rectangular spatio-temporal domain.

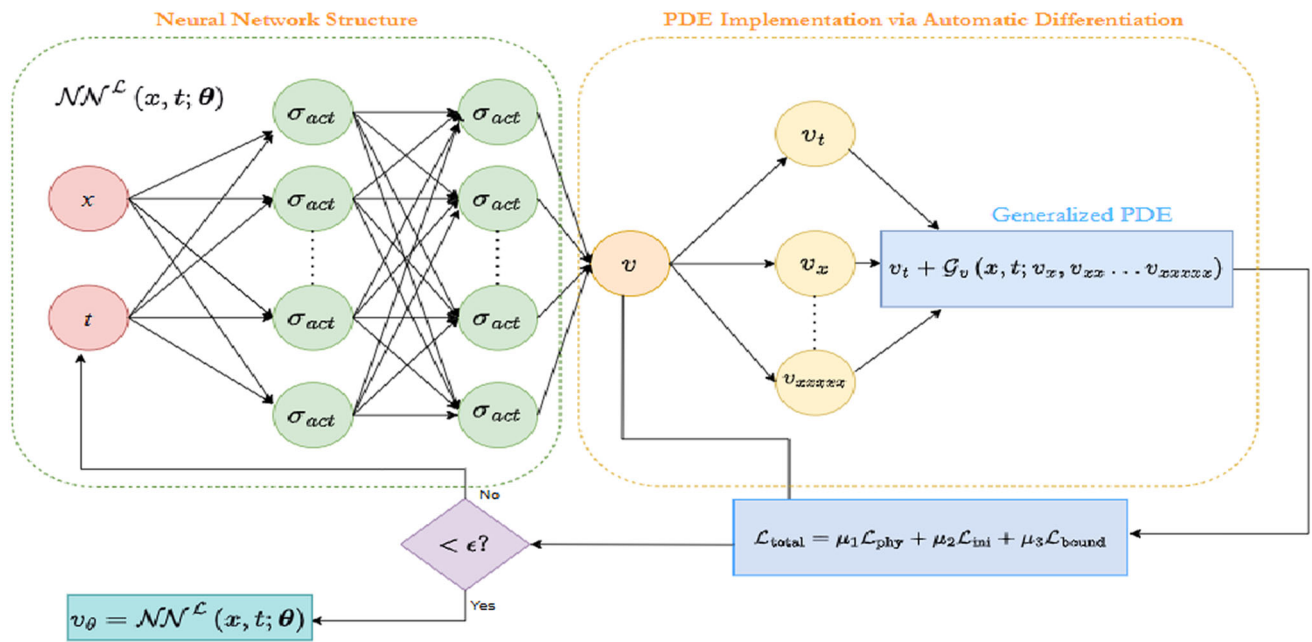


Fig. 1 Schematic diagram of PINN, highlighting automatic differentiation of PDE implementation and predicting the solutions with loss function

To offer extra clarification, various partial derivatives concerning v up to higher order with nonlinear terms are concluded in the function $\mathcal{G}_v$, whereas the fulfillment of partial derivatives as well as the solution v at boundary points is controlled with the governed boundary constraint $\mathcal{B}(v; x, t)$.

The main concept of the physics-informed neural network [34] is to utilize the deep neural network architecture to predict the solution as $\tilde{v}_\theta \approx v$. Here, θ symbolizes the combination of model parameters that are updated during the training of the neural network model. This PINN architecture is intended to fulfill partial differential equations naturally using boundary conditions while ensuring that all provided constraints are met.

$$v(x, t) \approx \tilde{v}_\theta(x, t) = \mathcal{NN}^{\mathcal{L}}(x, t; N_o, N_B, N_C; \theta). \quad (28)$$

Here, $N_o = \left\{ (x_i, t_i, I(v(x_i, t_i)))_{i=1}^{\mathcal{P}_o} \right\}$ represents the $\mathcal{P}_o$ training distributed samples during the initial phase at $t = 0$ of the training, $N_B = \left\{ (x_i, t_i, \mathcal{B}(v(x_i, t_i)))_{i=1}^{\mathcal{P}_b} \right\}$ symbolizes training samples of $\mathcal{P}_b$ on the boundary $\partial\mathcal{S}$, and $N_C = \left\{ (x_i, t_i, \mathcal{F}(\tilde{v}(x_i, t_i)))_{i=1}^{\mathcal{P}_c} \right\}$ is a set of $\mathcal{P}_c$ randomly assigned training points within $\mathcal{S}$ that imposes PDE physics on the neural network model. Also, $\theta = \{ \mathbf{W}_i^T, \mathbf{b}^i \}_{i=1}^{l-1}$ defines network weights and biases of the neural network.

The PINN technique integrates losses from the PDE (26) and boundary conditions (27) to restrict the neural network solution $\tilde{v}$. The total loss function $\mathcal{L}_{total}$ modifies the parameters of the neural network θ by assessing the weighted total of the residuals' $\mathcal{L}^2$ norms for both the PDE and the boundary conditions.

In the PINN framework, the physical constraints that are derived from the governing PDEs, which include conservation laws such as mass and momentum conservation, are embedded directly into the loss function. In our case, this is implemented by using the DeepXDE [35] library in Python with TensorFlow as the backend, where a custom PDE residual function is defined to represent the differential operators and the terms that capture the underlying physics. The solution is estimated using a neural network, and all required derivatives, such as spatial and temporal derivatives, are calculated via automatic differentiation by using TensorFlow. This approach

automatically traces the computational graph to calculate derivatives of any order, providing more accuracy and stability than standard numerical differentiation methods, especially for complicated PDEs with higher-order components. The total loss function is composed of multiple components:

$$\mathcal{L}_{\text{total}}(\theta; N_C, N_o, N_B) = \mu_1 \mathcal{L}_{\text{phy}}(\theta; N_C) + \mu_2 \mathcal{L}_{\text{ini}}(\theta; N_o) + \mu_3 \mathcal{L}_{\text{bound}}(\theta; N_B), \quad (29)$$

where

$$\begin{aligned} \mathcal{L}_{\text{phy}}(\theta; N_C) &= \frac{1}{\mathcal{P}_c} \sum_j \|\tilde{v}_t + \mathcal{G}_v(x_i, t_i; \tilde{v}_x, \tilde{v}_{xx}, \tilde{v}_{xxx}, \dots)\|_2^2, \\ \mathcal{L}_{\text{ini}}(\theta; N_o) &= \frac{1}{\mathcal{P}_o} \sum_j \|I(\tilde{v}(x_i, t_i); x_i, t_i)\|_2^2, \\ \mathcal{L}_{\text{bound}}(\theta; N_B) &= \frac{1}{\mathcal{P}_b} \sum_j \|\mathcal{B}(\tilde{v}(x_i, t_i); x_i, t_i)\|_2^2 \end{aligned}$$

Here, μ_1 , μ_2 , and μ_3 are the network hyperparameters that are used to customize the network weights. Optimizing hyperparameter values improves the equilibrium of partial losses during training. The total loss function $\mathcal{L}_{\text{total}}$ of the neural network is defined as the linear combination of three different losses, i.e., $\mathcal{L}_{\text{phy}}(\theta; N_C)$, $\mathcal{L}_{\text{ini}}(\theta; N_o)$, and $\mathcal{L}_{\text{bound}}(\theta; N_B)$. The primary residual loss $\mathcal{L}_{\text{total}}(\theta, N_C)$ requires the neural network to follow the physics of the provided PDE, whereas the neural network parameters are optimized with the help of loss terms $\mathcal{L}_{\text{ini}}(\theta; N_o)$ and $\mathcal{L}_{\text{bound}}(\theta; N_B)$ by assuring the fulfillment of the initial and boundary conditions of the PDEs. Proper hyperparameter selection is critical for minimizing the total loss function when training the model. It improves both accuracy and efficiency in predicting the intended result of a certain challenge. Furthermore, the residual loss term provides insight into how effectively the neural network model performs inside its domain and its ability to anticipate actual outputs.

4 Genetic programming based on evolutionary algorithm

Evolutionary algorithms (EAs) are a type of optimization algorithm influenced by natural selection and evolution [36]. It integrates machine learning techniques to enhance convergence speed as well as optimization performance with great efficiency. The machine learning model extracts patterns from data to guide search tactics, whereas evolutionary algorithms efficiently explore diverse solution spaces with recognition of different patterns. Nowadays, EAs have become very popular in computational algorithms as it is commonly utilized in solving various complex scenarios due to their capacity to replicate natural evolution and execute population-based random searches [37, 38]. Moreover, evolutionary algorithms are stochastic search methods that imitate natural evolution. It cannot guarantee that optimality may be achieved while dealing with complex real-time problems, but population based yields an estimated optimum solution suitable for many application contexts [38]. EAs generally use several techniques that include selection, evolution, and iterative population updates to explore and develop optimization solutions [36]. Figure 2 demonstrates different operations of the evolutionary algorithm from initialization to optimal solutions.

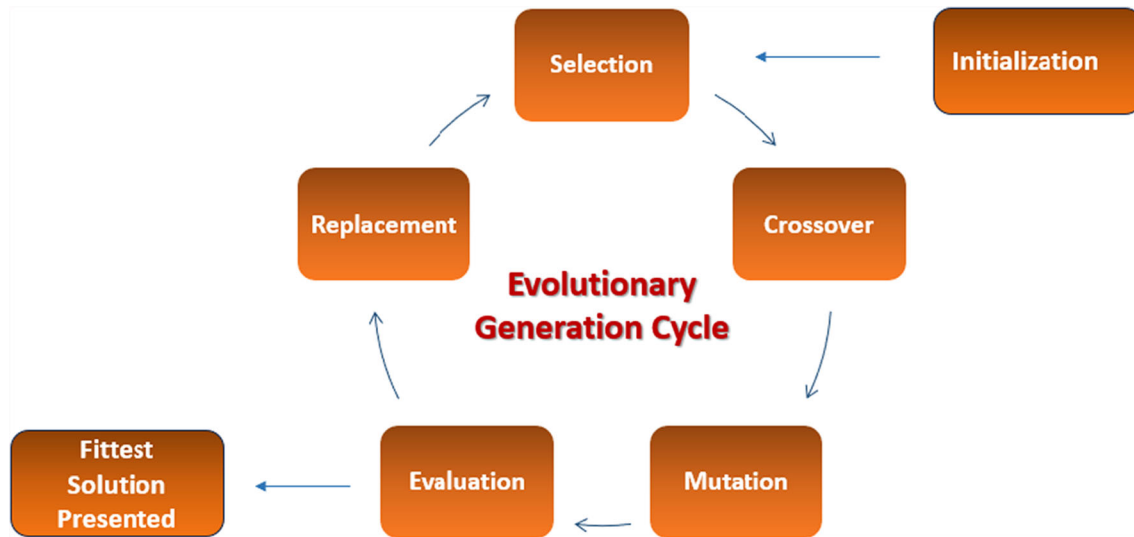


Fig. 2 Evolutionary cycle of a standard evolutionary algorithm. Each block represents an operation on a set of possible solutions

4.1 Genetic algorithm

Genetic algorithms (GAs) represent a category of evolutionary algorithms (EAs) that emulate the principles of the natural selection process to address optimization and search challenges [39]. This is a population-based search algorithm inspired by the biological progression of living organisms. New populations are generated through the repeated application of genetic operators on the individuals within the existing population. The essential components of GAs include chromosome representation, selection, crossover, mutation, and the calculation of the fitness function [39, 40]. A chromosome comprises a set of values that fully defines a potential solution throughout the optimization procedure. The selection process involves identifying elite individuals from the current population to serve as parents for the next generation. Fitness values are utilized as the criteria to determine which individuals qualify as elite [37]. The crossover operator generates two new offspring by selecting and duplicating specific bits from each of the two parent strings, and mutation in genetic programming acts as an evolutionary operator that introduces diversity among the population of potential solutions [39, 40]. The fitness function evaluates the quality of the solutions produced in genetic algorithms and is modeled as a part of the overall optimization technique. It serves as an objective metric and guides the evolutionary search process by choosing the most successful individuals for reproduction in succeeding generations [38, 39]. Furthermore, the fitness function plays an important role in assigning the numerical scores to each solution based on its performance. A higher fitness value increases the probability of individuals being selected for crossover and mutation. Therefore, genetic algorithms are more suitable and less expensive in predicting the desired relationships that best fit the data in computational methods. Moreover, this technique is highly adaptive to their environment and very helpful in finding the behavioral patterns of complex, noisy data of real-world problems.

4.2 Symbolic regression

In recent years, neural networks (NNs) have become a very emerging tool in machine learning algorithms to solve many complex mathematical as well as real-world problems. PINNs are one kind of deep learning architecture that has gained popularity among researchers to predict the solutions of many complex nonlinear PDEs. While it is highly effective, a major limitation of PINNs is their difficulty in interpretation, which often leads to their perception as black-box solvers. In the field of machine learning-based predictions, symbolic regression (SR) has

Fig. 3 Three main components of evolutionary computing—symbolic regression (SR), genetic programming (GP), and evolutionary algorithms (EAs)

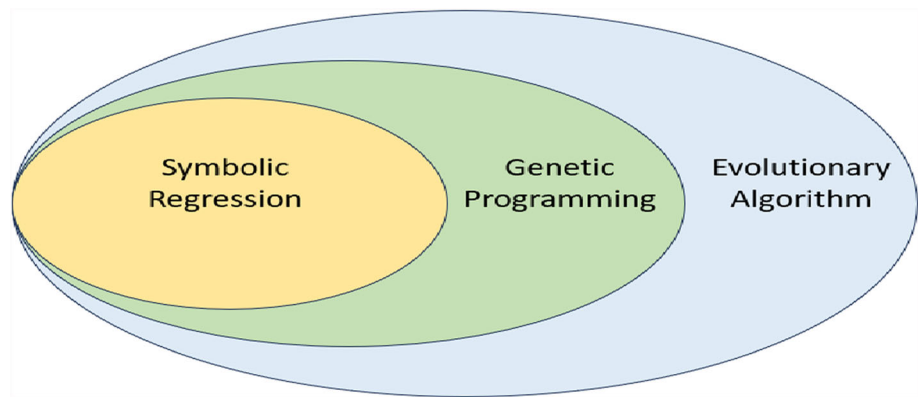
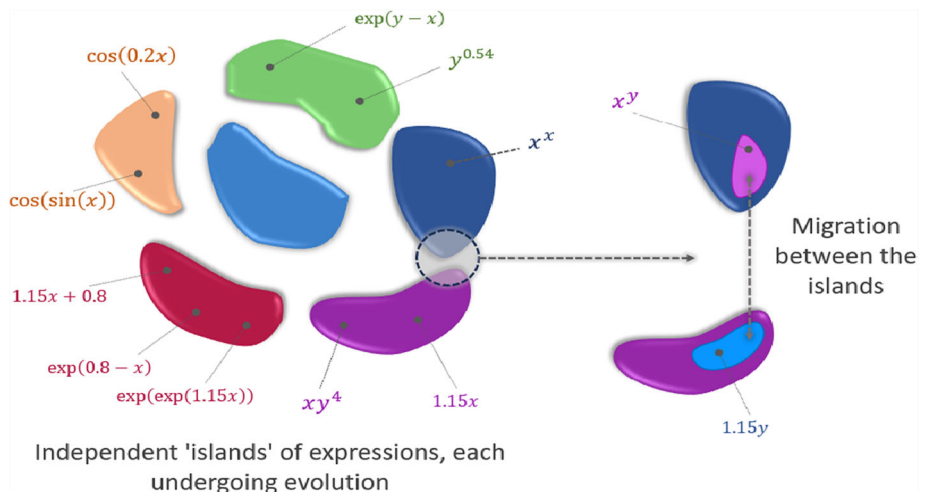


Fig. 4 In genetic programming, different populations evolve independently on separate islands, and after a set number of generations, individuals migrate between islands to share improvements



captured the interest of numerous researchers because it effectively finds analytical formulations for complicated systems of PDEs [21]. In this study, we have also used this methodology to acquire mathematical expressions for predicted solutions of our complex evolutionary PDEs.

SR is a genetic programming (GP) approach based on evolutionary algorithms in modern machine learning techniques. Figure 3 illustrates the relationship between SR and GP, and the broader family of evolutionary algorithms (EAs). As the diagram shows, both SR and GP represent relatively small, specialized subsets within the expansive landscape of evolutionary algorithms. When PINN algorithms are applied to a dataset and predict some solutions, this SR model is used to discover the analytical expressions that best fit the experimental as well as predicted data with high proficiency. The mathematical expressions of predicted performance can be viewed as a tree structure [18]. In this tree-based structure, the terminal node signifies the variables and constants, whereas the intermediate cells symbolize several mathematical operations that include addition, multiplication, subtraction, and division. Symbolic regression uses these fundamental mathematical operations, as well as basis functions like exponential and trigonometric functions, to identify underlying correlations [20]. A SR model is built by combining operators and coefficients in different ways, creating endless possibilities to explore [41]. The mathematical operator domain is bounded by a finite number of operations and a maximum model complexity criterion. Finding the right model requires an efficient search. In this context, genetic programming is an effective technique for symbolic regression to find the search space. Genetic algorithms evolve models by continuously improving them based on how well they perform according to a given fitness function. This fitness function helps to pick the models that are most likely to perform well. Thereafter, the models evolve by mixing parts from different models (i.e., crossover) and making small random changes (i.e., mutation) to create new candidate

Fig. 5 A flowchart of the symbolic regression algorithm shows how it explores and evolves equations to uncover the best-fitting, interpretable relationships hidden within the data

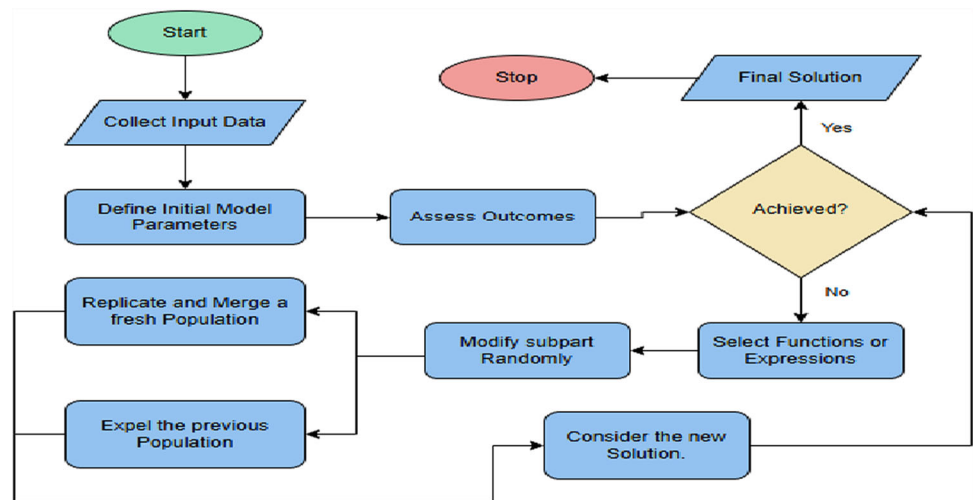
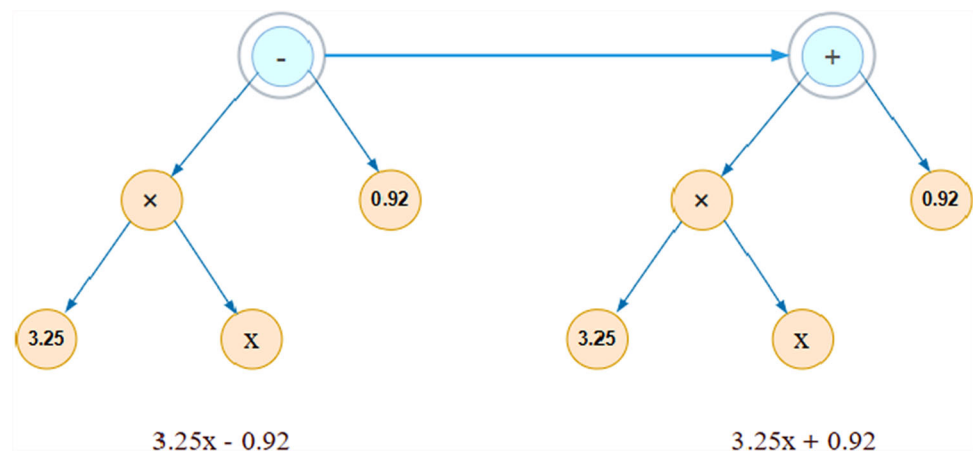


Fig. 6 Highlighting mutation operation that modifies portions of a mathematical expression in a genetic algorithm to investigate novel solutions and enhance overall performance



models [20, 40]. At the same time, natural selection removes candidates with the lowest fitness from the population. The use of iterative approaches to explore the solution space is determined by a mix of randomness and precise processes for fitness evaluation, crossover, and mutation.

The fitness function typically measures the discrepancy between experimental results and model forecasts and frequently utilizes absolute or squared error metrics. Additionally, the minimum squared error in the fitness function indicates a superior alignment of the expressions with the experimental data. Figure 5 represents a flowchart of symbolic regression based on a genetic algorithm, highlighting the steps of population selection, replication, and merging of a new population, and outcome evaluation. Let us consider a mathematical expression, $g(x) = \cos(2.1x) + 6e^{-20x}$. This can be viewed as a tree structure, where the “+” operator acts as a root of the tree, whereas $\cos(2.1x)$ and $6e^{-20x}$ represent two branches of the tree. With a defined collection of basis functions and operators, we can conceptualize randomly generated functions as computational programs that produce predictions for a target variable. These programs incorporate unspecified parameters, including coefficients, that are modified throughout the optimization process. The effectiveness of each function is assessed through its “fitness,” which is generally quantified by calculating the sum of squared differences between the predicted values and the actual data points. Mutation and crossover are two essential operations that are employed for refining the functions in the optimization process. From Fig. 6, we can see the tree structure of the mutation operation in the genetic algorithm. In Fig. 6, we observe a symbolic mutation where the original mathematical expression, written as $3.25x - 0.92$, undergoes a subtle yet meaningful transformation. Specifically, the

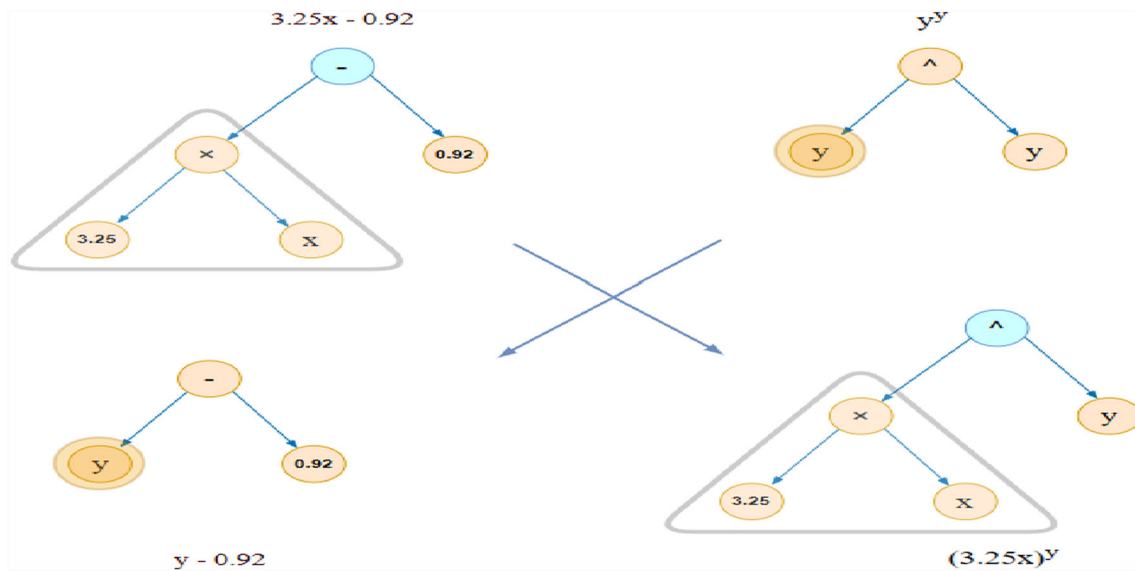


Fig. 7 Crossover mutation in genetic programming swaps parts of two parent programs to create new, potentially better-performing offspring solutions

subtraction operator is replaced with an addition operator, resulting in the expression $3.25x + 0.92$. This mutation illustrates how even a minor symbolic change in the structure of a symbolic equation can lead to a completely different interpretation or behavior in the model's output. The crossover operation combines components of two parent functions to form a new function, and it is generally known as “offspring.” In Fig. 7, we can observe a clear example of crossover mutation taking place between two parent functions as part of the symbolic regression process. The original parent functions involved are $g_1(x) = 3.25x - 0.92$ a linear expression, and $g_2(y) = y^y$, a nonlinear self-exponential function. During the crossover event, portions of each parent's expression are recombined to generate two new offspring functions. The first resulting function, $g_3(y) = y - 0.92$, emerges by combining the structure of $g_2(y)$ with the constant term from $g_1(x)$. The second offspring, $g_4(x, y) = (3.25x)^y$, blends the linear coefficient from $g_1(x)$ with the exponential structure of $g_2(y)$, producing a new function that now depends on both variables. This example highlights how crossover mutations can introduce novel and potentially more expressive symbolic forms by recombining building blocks from existing expressions.

Therefore, this study introduces a hybrid methodology that integrates PINNs with genetic programming to obtain analytical solutions for nonlinear PDEs. The predicted solutions from the PINN have been utilized as input data for symbolic regression within the GP framework. PINNs present the advantage of resolving PDEs without the need for substantial labeled datasets, as they embed the physical laws directly into the loss function during the training phase. On the contrary, genetic programming is proficient in revealing symbolic relationships in data, producing expressions that are understandable and interpretable by humans. This strategy leverages the analytical clarity of genetic programming alongside the data efficiency and flexibility of PINN by integrating these two techniques. This approach enhances both the precision and applicability of PDE solutions while offering insights into the fundamental dynamics of the system via concise analytical representations. This makes it especially beneficial for theoretical examination, model verification, and additional scientific interpretation.

5 Overview of nonlinear partial differential equations (PDEs)

5.1 Governing PDEs

In this work, we have derived some nonlinear PDEs that arise in the modeling of arterial blood flow. These equations are highly complex and play a significant role in the study of blood flow dynamics in arteries, where dispersion effects and nonlinear wave interactions critically influence blood flow propagation. In Eq. (24), the nonlinear convection term vv_x illustrates the mechanism of how blood flow propels itself throughout the large arteries. Additionally, the term $\frac{1}{2}v_{xx}$ represents a diffusion component and accounts for the viscous effects in the blood flow dynamics. Furthermore, Eq. (25) is a fifth-order evolutionary equation and can be said a higher-order extension of Eq. (24), including third-order (v_{xxx}) and fifth-order (v_{xxxxx}) dissipative terms. These higher-order partial derivative terms help to account for the wave dispersion effect as well as enhance stability and regulate wave steepening, therefore assisting in the characterization of arterial pulse wave propagation over long distances. Again, in both the equations (i.e., Eqs. (24) & (25)), $\frac{M}{2}v$ refers to the external forcing term, generally related to arterial external pressure or wall elasticity.

The discovered symbolic expressions of the solutions of nonlinear PDEs in this study show patterns that align with well-known hemodynamic behaviors, including nonlinear advection, diffusion, and higher-order dispersive effects. Coefficients tied to first and second order derivatives often relate to aspects like flow velocity and viscous dissipation, while higher-order terms appear to reflect the elasticity of vessel walls and the propagation of pressure waves. Nonlinear interactions such as vv_x suggest convective acceleration, which is commonly observed in pulsatile blood flow. Connecting these terms and parameters with physical quantities such as viscosity, vessel compliance, and pressure gradients can help bridge the gap between data-driven discoveries and physiological understanding. Relating these coefficients to parameters found in classical models or observed experimentally could enhance our understanding of their physiological significance.

5.2 Importance of analytical solution

Analytical solutions of nonlinear PDEs play a fundamental role in understanding the characteristics and behavior of the equations. Nonlinear PDEs are frequently difficult to solve accurately, but when analytical solutions are accessible, they offer some major benefits. It has a deep insight into physical behavior. Analytical solutions produce accurate expressions for the dependent variables that help to understand the qualitative behavior, such as wave formation, propagation, or dissipation of the system. It identifies important phenomena, including shock production, soliton dynamics, and wave breaking. Moreover, analytical solutions of nonlinear PDEs reduce computational costs by quickly evaluating the various values of the parameters without executing complicated simulations. This is commonly useful in sensitivity analysis, control, and optimization problems.

Furthermore, mathematical formulations of the solution help to increase mathematical understanding by highlighting the symmetry aspects of PDEs and classifying them based on their solution types and solvability. It has a major impact on machine learning models, as analytical expressions are used to guide or train the data-driven machine learning models. Therefore, the analytical solutions of the PDEs in arterial blood flow modeling are critical in understanding the behavioral pattern of the waves and validating numerical simulations. It plays a crucial role in a variety of medical applications that include identifying illnesses in the arterial system and managing blood pressure pulses with medical devices to improve therapy.

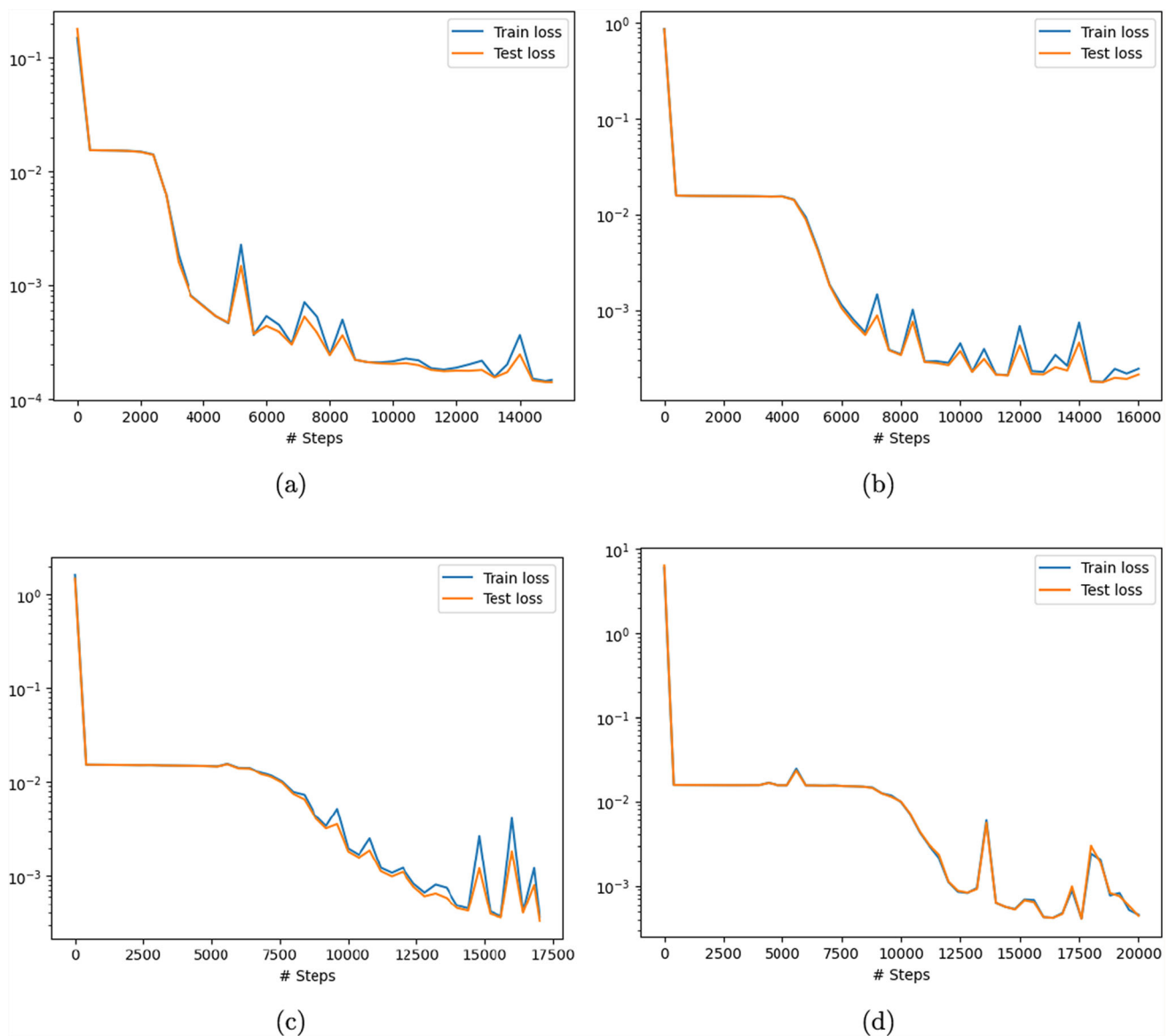


Fig. 8 Visualization of train loss and test loss over epochs for forced Burger's equation with four different M values. Here, x -axis represents the number of epochs (or, steps), and y -axis represents the loss values during the training of the model for various values of M : **a** loss curve for $M = 0.2$, **b** loss curve for $M = 0.4$, **c** loss curve for $M = 5$ **d** loss curve for $M = 10$

6 Results and discussions

This section provides a concise result of our research in identifying the solutions of nonlinear PDEs. A PINN model architecture was employed, and its predicted outputs were used to derive approximate analytical solutions through a symbolic regression model. Additionally, the PINN model's performance was evaluated during training across various values of the magnetic parameter M . Furthermore, a comprehensive sensitivity analysis of the SR model's hyperparameters was performed to understand how different parameter settings influence the accuracy and robustness of the obtained analytical solutions.

Table 1 PINN hyperparameters and performance results for the forced Burgers' equation at varying M , showing the impact of different values of M on training configuration and accuracy

PINN hyperparameters for forced Burgers' equation				
Parameter	$M = 0.2$	$M = 0.4$	$M = 5$	$M = 10$
Learning rate	1e-3	1e-3	1e-3	1e-3
Optimizer	Adam	Adam	Adam	Adam
Hidden layers	5	5	5	5
Neurons per layer	30	30	30	30
Activation function	sin	sin	sin	sin
Training epochs	14,000	16,000	17,500	20,000
Performance metrics:				
Training loss	1.79×10^{-4}	1.79×10^{-4}	3.61×10^{-4}	4.11×10^{-4}
Test Loss	1.78×10^{-4}	1.78×10^{-4}	3.29×10^{-4}	4.09×10^{-4}

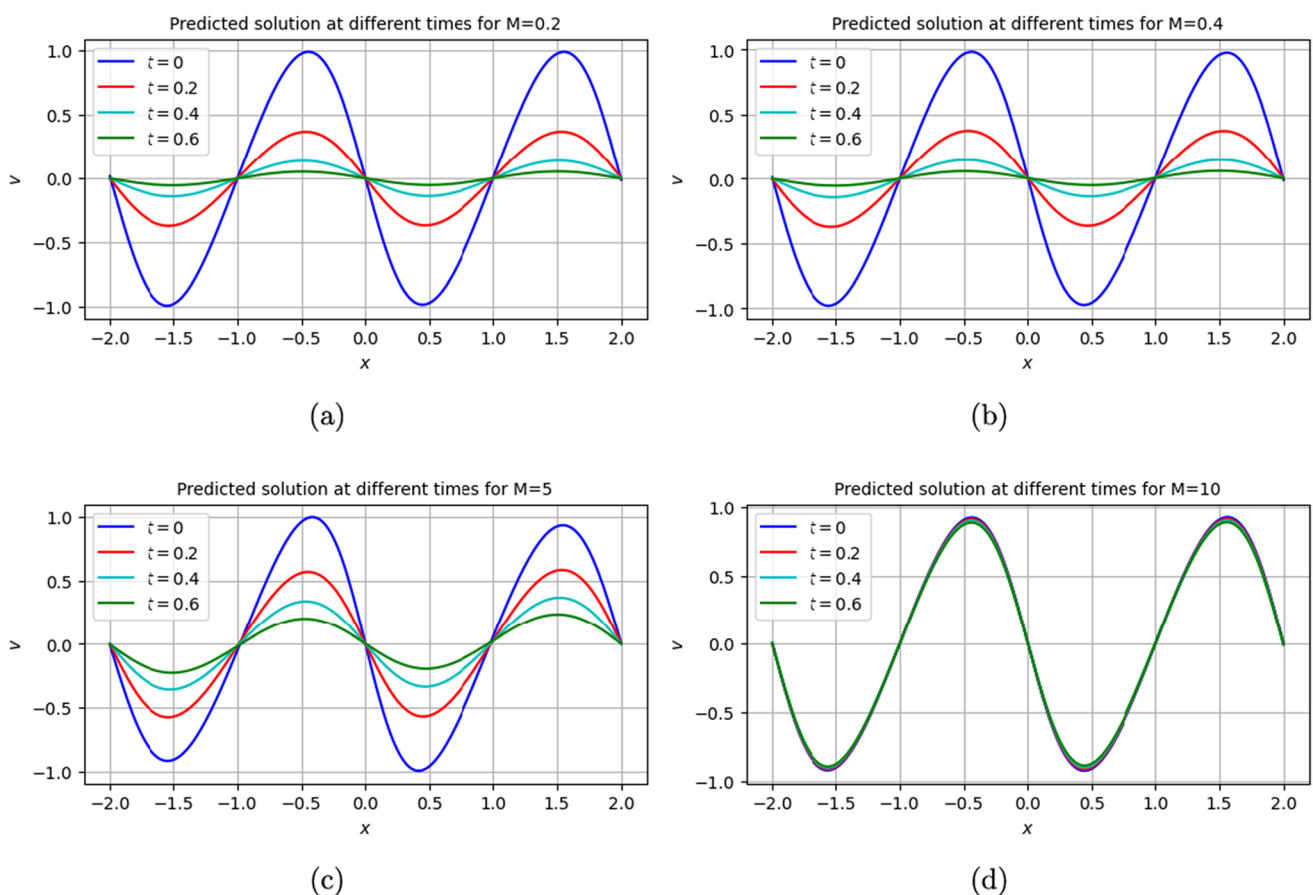


Fig. 9 Illustration showcases the solutions of the forced Burgers' equation across different time scales, clearly demonstrating how well the PINN model satisfies the boundary conditions for a range of values of the parameter M

6.1 Physics-informed neural network (PINN)-based predictions and analysis of the results

In this work, physics-informed neural network (PINN), a machine learning algorithm, has been used to find the predicted solution of forced Burgers' equation (Eq. 24) and fifth-order evolutionary equation (Eq. 25). Various configurations of hidden layers and neurons were empirically tested to identify a structure that provided a good balance between accuracy and training stability. Although no formal optimization strategy was used, the chosen

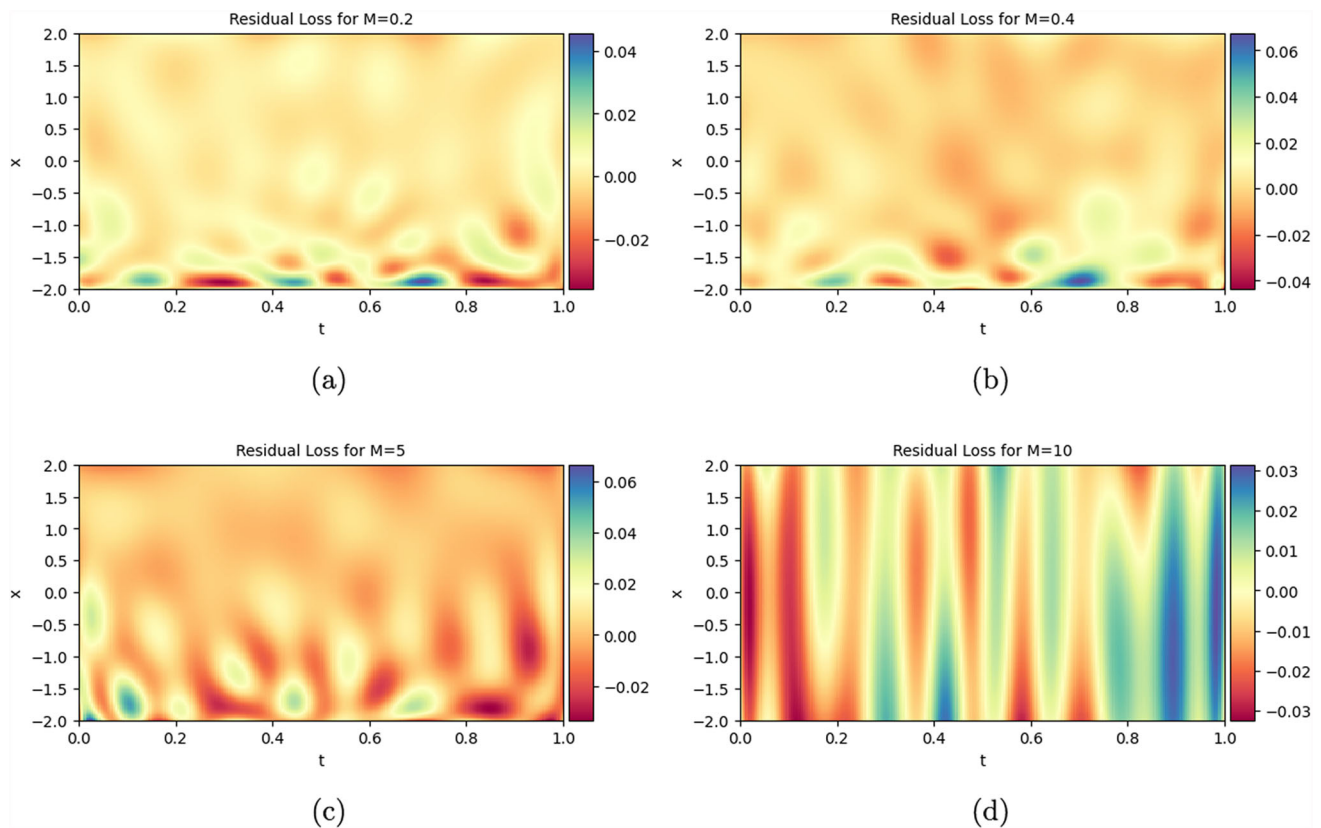


Fig. 10 Image illustrates the spatial distribution of residual losses across the rectangular domain for the forced Burgers' equation, highlighting how these losses vary under different values of the parameter M

architecture aligns with standard practices and was adequate for the complexity of the problem. To solve Eq. (24) for different values of M , the deep neural network was created with 5 hidden layers and 30 neurons in each layer. In this instance, the sine activation function was selected to update the network weights during the back propagation. Following the initial setup, the model was trained across various numbers of iterations and a range of M values to explore how different configurations would influence its performance and convergence behavior. We set the initial pulse wave as $v(x, 0) = -\sin(\pi x)$ and boundary condition as $v(-2, t) = v(2, t) = 0$, where x varies from -2 to 2 , and t ranges from 0 to 1 . The PINN model was trained using 2500 points within the domain, and 1500 additional points were used to evaluate the model's performance. Additionally, 700 and 600 data points were assigned to enforce the boundary and initial conditions, respectively, during the training process. It is crucial to ensure that the boundary conditions are satisfied with high accuracy when training a PINN model, especially because any significant residual loss at the boundaries can lead to inaccuracy or physically inconsistent solutions. Paying careful attention to these constraints helps maintain the integrity of the model and reinforces its ability to learn solutions that truly reflect the underlying physics. Sometimes, PINN models provide good residual losses without satisfying the proper boundary conditions, but this leads to inappropriate predictions of the results. From Fig. 8, throughout the training process, we observed a clear and consistent decrease in the value of the loss function over successive iterations. This steady decline is a strong indication that the model is effectively learning from the data, gradually minimizing its error and improving its performance as training progresses.

The parameter M in Eq. (24) has a major impact on achieving the low residual losses in its domain. Lower M values do not need an excessive number of epochs (or iterations) during the training of the model, whereas higher M values need a large number of iterations to successfully train the PINN model. Figure 8a shows that the PINN model was trained for 15,000 iterations to minimize the loss during the training process for the equation with $M = 0.2$. Again, from Fig. 8b, it is clear that 16000 epochs were needed to achieve the train loss and test loss as

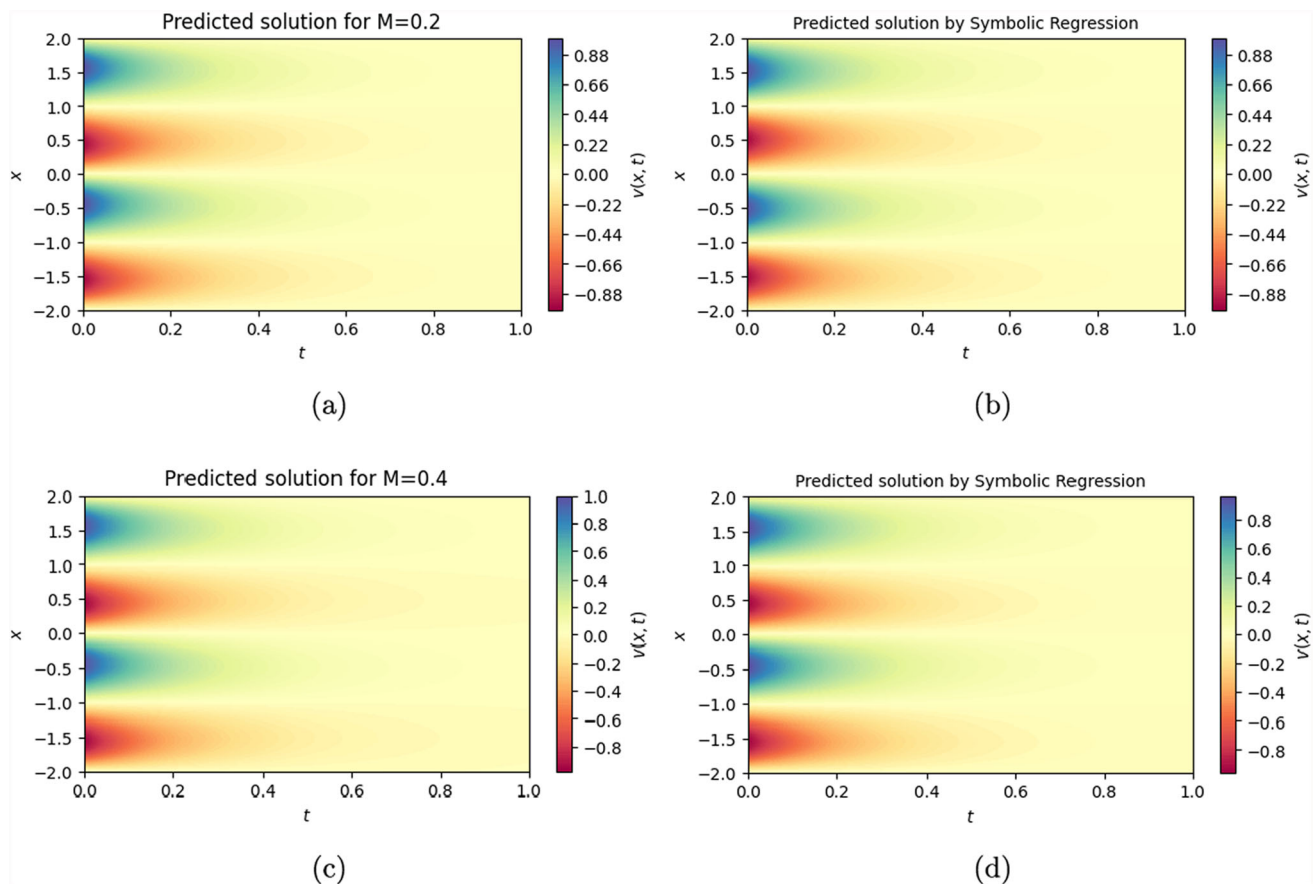


Fig. 11 Visualization of PINN solution and symbolic regression solution of forced Burgers' equation for different M values: **a** PINN prediction for $M = 0.2$, **b** symbolic solution for $M = 0.2$, **c** PINN prediction for $M = 0.4$, **d** symbolic Solution for $M = 0.4$

1.79×10^{-4} and 1.78×10^{-4} , respectively, for $M = 0.4$. Similarly, it is evident from Fig. 8c and d that for the M values of 5 and 10, iterations of 17,000 and 20,000 were selected, respectively, to achieve low loss values. In that large value of M , the train loss and test loss were 3.61×10^{-4} , 3.29×10^{-4} and 4.11×10^{-4} , 4.09×10^{-4} , respectively, with the same model configurations. The training and testing losses corresponding to various PINN configurations are systematically presented in Table 1, highlighting the performance across different values of M . Again, it can be noticed in Table 1 that for large values of M with an increased number of epochs, the train losses and test losses have slightly greater values than lower M values. These results represent low loss values across the domain. To achieve similarly lower loss values with higher values of M , it is necessary to increase the number of training iterations or to adjust network parameters such as the learning rate, number of hidden layers, and the number of neurons per layer accordingly. Figure 9a–d illustrates the solution curves at different time scales and demonstrate how well the boundary conditions are satisfied by using the PINN model for different values of the parameter M .

From Fig. 10a–d, the rectangular regions of residual losses can be visualized to assess the model's performance across the respective domain. As it is seen from these plots, the maximum residual loss value for $M = 0.2$, $M = 0.4$, $M = 5$, and $M = 10$ are 0.04, 0.06, 0.06, and 0.03, respectively, and these values are significantly low. Again, we have calculated the mean residual error of these models. We have got the mean residual errors as 0.003, 0.005, 0.006, and 0.01 for respective PINN models with specific M values as indicated earlier. These minimum mean residual errors signify that the PINN models have performed well with high accuracy and better precision.

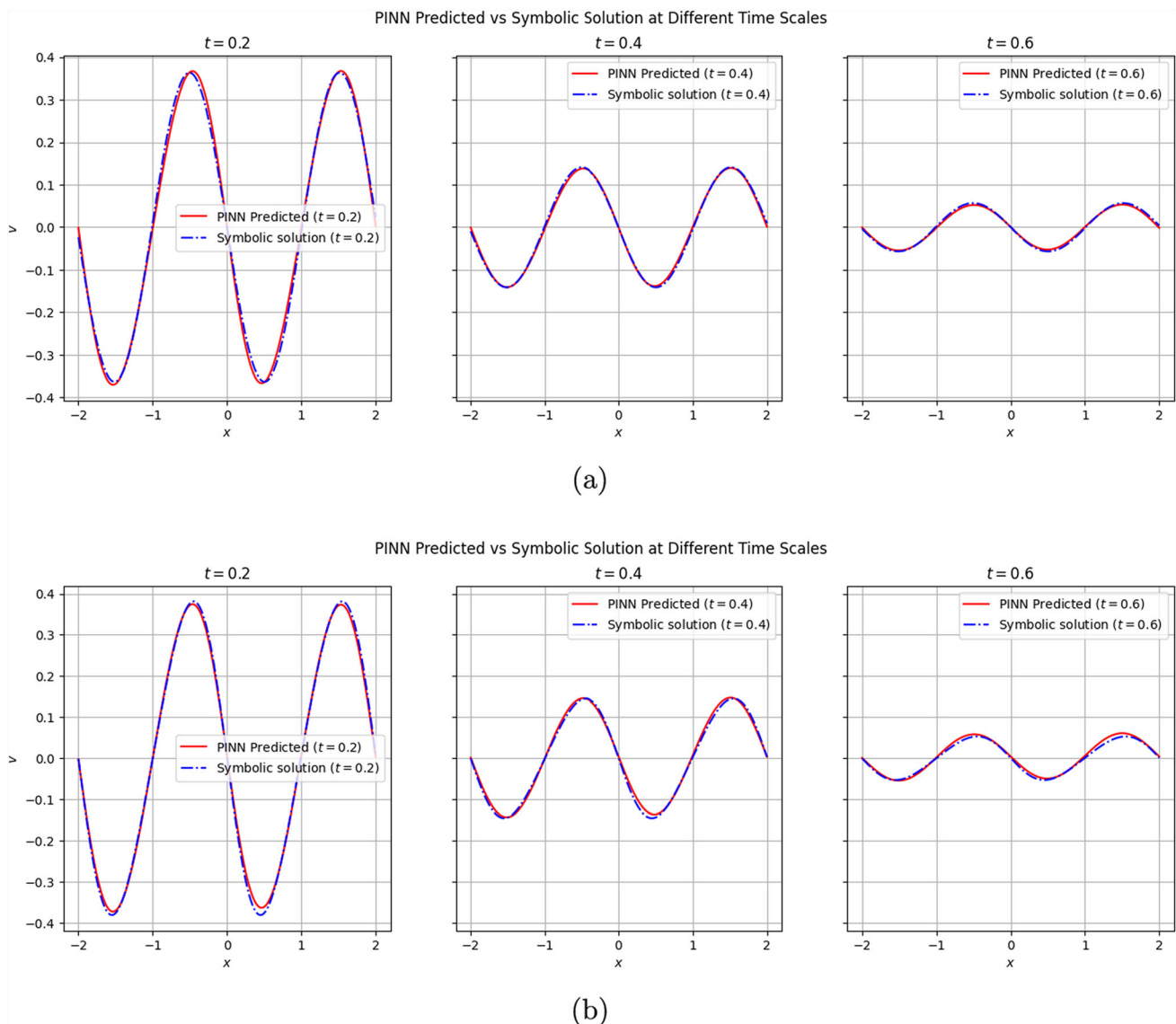


Fig. 12 Comparison of PINN-based prediction and symbolic regression solutions of forced Burgers' equation for different time scales with distinct small M values: **a** solution comparison for $M = 0.2$, **b** solution comparison for $M = 0.4$

6.2 Analytical solution by using symbolic regression for forced Burgers' equation

Symbolic regression has emerged as a powerful and intuitive approach for discovering compact, interpretable expressions that approximate the solutions of partial differential equations. It provides a data-driven substitute for conventional numerical techniques by directly learning mathematical connections from data and enables researchers to find beautiful formulations that might otherwise remain undetected and obtain a deeper understanding of the underlying physics. We apply PySR [42], a Python and Julia module for scientific symbolic regression, to discover the best-fitting expressions of PINN-based solutions. This technique was performed after utilizing the PINN model. Here, the raw data of x and t with predicted values of the PINN model have been used to find the best-fitting curve of the predicted solutions with varying M parameters. In the symbolic regression model, the iteration was set to 27, whereas the population size was 30. Again, the batch size was set to 264 to enhance the training as well as the convergence speed. In the PySR model, binary operators such as addition, multiplication, subtraction, and division were used, whereas exponential, cosine, and sine functions were

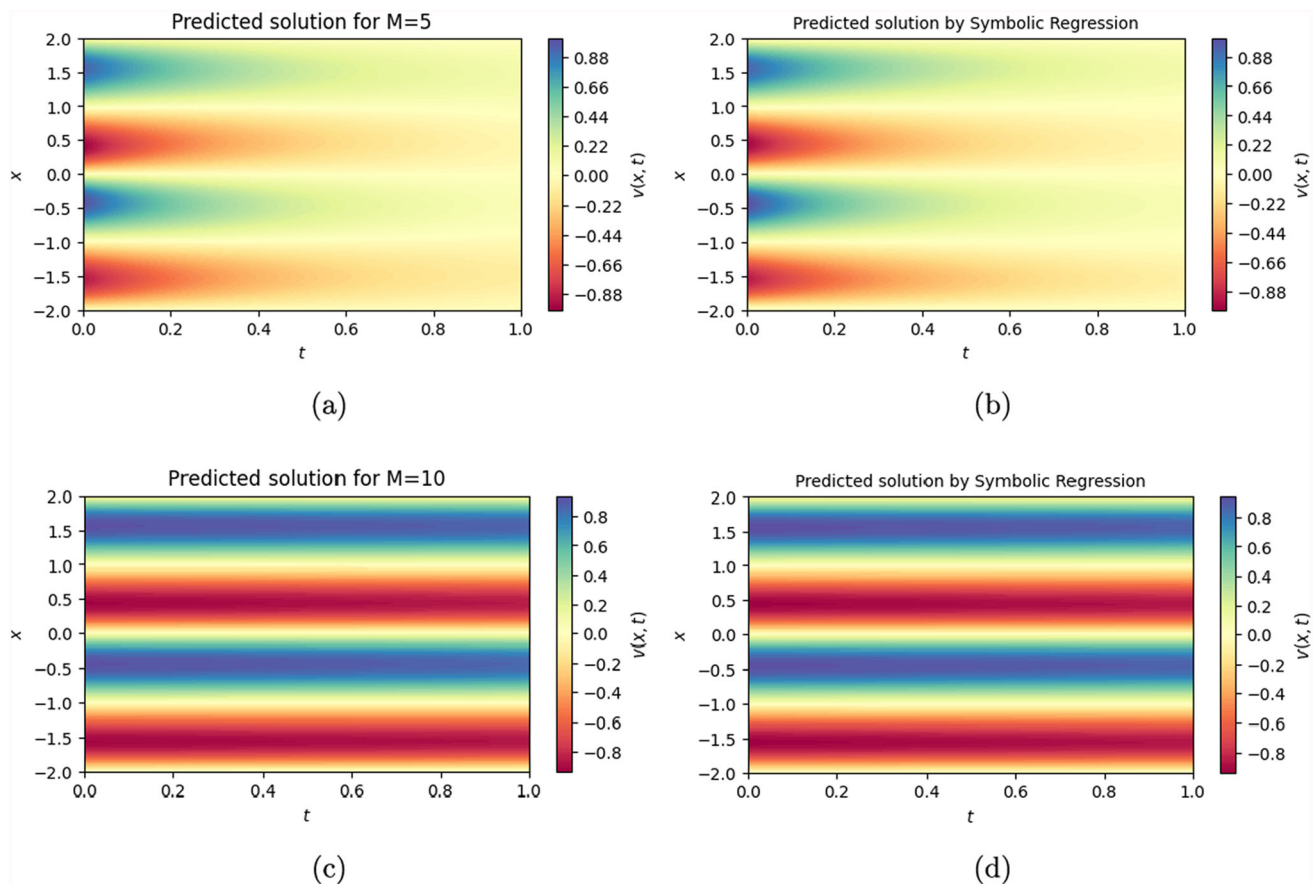


Fig. 13 Visualization of the solutions obtained from the PINN and symbolic regression for the forced Burgers' equation across various values of M : **a** PINN prediction at $M = 5$, **b** symbolic solution at $M = 5$, **c** PINN prediction at $M = 10$, **d** symbolic solution at $M = 10$

considered for unary operators. The selected functions were chosen to accurately represent the anticipated physical or analytical characteristics of the solution, while ensuring adequate expressiveness to capture nonlinear dynamics. The inclusion of binary operators facilitates the formulation of algebraic expressions composed of two operands. When the SR model is trained, it provides the best mathematical expressions that describe the relationship between the predicted data and input features such as x and t . The identified equation brings a prediction error or Mean Squared Error (MSE) loss term with a specific complexity value. A lower MSE loss value means the mathematical expression is a good fit with the provided data, and complexity defines how many binary and functional operations have been used in that analytical expression.

Two-dimensional (2D) plots from Figs. 11a–d help to visualize the symbolic regression performances with the PINN solution throughout the domain. Moreover, from these plots, it is evident that solutions of the symbolic regression model encapsulate the exact characteristics of PINN predictions. Again, from Fig. 12a and b, it is evident that symbolic regression effectively fits the predicted data generated by the PINN model. After performing the SR model, the symbolic solution for $M = 0.2$ is obtained as:

$$v(x, t) = e^{t(-3.76)} \sin\left(\frac{x}{-0.322}\right) (-0.321) \frac{1}{(-0.797)(t \cdot 0.560 + 0.411)} \quad (30)$$

Here, the mean absolute error between the PINN and the SR solution is 0.008, which is a significantly lower value

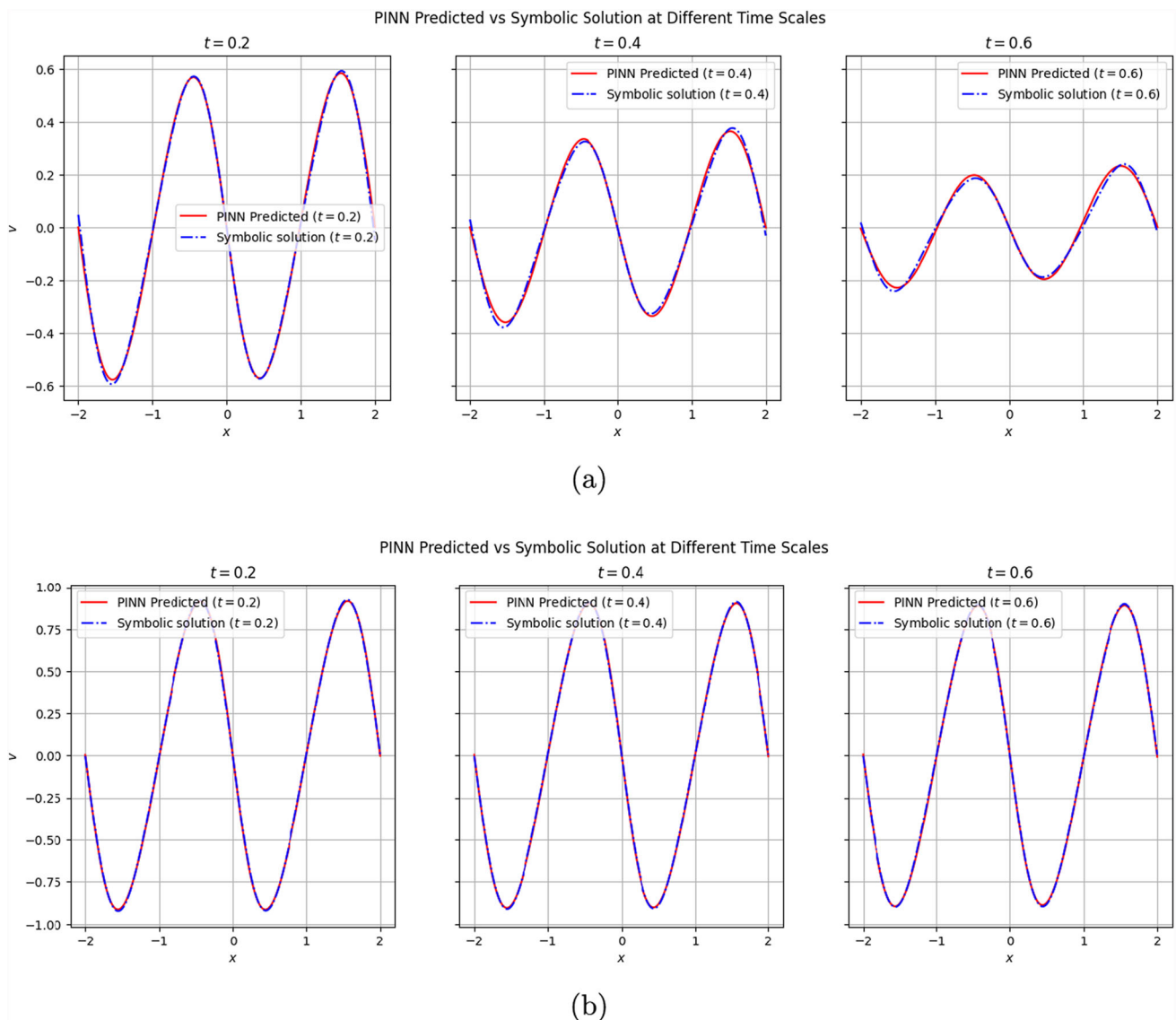


Fig. 14 A comparison of predictions derived from PINN and symbolic regression methods for the forced Burgers' equation across various time scales with distinct large values of M : **a** comparison of solutions for $M = 5$, **b** comparison of solutions for $M = 10$

Table 2 Quantitative comparison of performance metrics for different values of M in the context of the forced Burgers' equation

Loss metrics for forced-Burgers' equation				
Loss type	$M = 0.2$	$M = 0.4$	$M = 5$	$M = 10$
Mean residual loss	0.003	0.005	0.006	0.01
MSE loss of symbolic regression	0.0002	0.00005	0.0002	0.00005
Mean absolute error between PINN and SR model	0.008	0.005	0.01	0.006
Complexity of discovered equation	19	25	22	17

The metrics include the mean residual loss of the PINN model, the MSE loss of the symbolic regression output, the mean absolute error between the PINN and symbolic regression predictions, and the complexity of the discovered symbolic expressions

and performs very well in finding the best-fitting curve. For the mathematical expression (Eq. 30), the SR model

yields an MSE loss of 0.0002 with a corresponding complexity of 19. This small loss value indicates the best-fitting relationships with the data. Again, for the value of $M = 0.4$, the mathematical expression that is found by the SR model is expressed as:

$$v(x, t) = \cos(t) \sin(x \cdot 3.14 + \cos((x + 0.0892 + 2.43)3.09)(-0.158))(-0.957) \frac{1}{e^{t \cdot 4.50}} \quad (31)$$

For this discovered equation, the mean absolute error between the PINN and SR solutions is 0.005. This small error value indicates a highly accurate and reliable mapping between the input and output data. It demonstrates that the SR model has captured the underlying relationship exceptionally well, reflecting both precision and robustness in its performance. Furthermore, the derived analytical expression achieves a notably low MSE of 0.00005, which highlights excellent alignment with the reference data. With a symbolic complexity of 25, the model also retains a manageable level of interpretability, balancing accuracy with simplicity.

Furthermore, for the value of $M = 5$, the number of iterations was set to 28 with a population size of 30, and the batch size was the same as for small M values (i.e., $M = 0.2$ and $M = 0.4$). But, for the value of $M = 10$, the iteration was 25. It is visible from Fig. 13a–d that the larger M values have a significant effect on the solution of Eq. (24). The SR model has successfully discovered the analytical expressions for these large values of M . Sometimes, the population size and number of iterations need to be adjusted accordingly to obtain the optimal mathematical expressions that most suit the given data in the SR model.

Figure 14a and b compares the solution graphs for different time scales with larger M values. The predicted solution of the SR model has very minimal deviation from PINN-predicted values, offering an accurately fitted relationship between the input and output data. The mathematical solution that the SR model has found for $M = 5$ is expressed as:

$$v(x, t) = \frac{0.776 \sin\left(\frac{x}{-0.315}\right)}{e^{t \cdot 2.55} \cos(\sin(t \cdot 0.474 - \cos(x(-1.49))))} \quad (32)$$

In this case, the mean absolute error (MAE) between the PINN solution and the symbolic regression expression is remarkably low, which is just 0.01. This indicates a very close agreement between the two methods. Furthermore, the MSE loss associated with Eq. (32) is only 0.0002, and the corresponding expression complexity is 22. Again, for $M = 10$ in Eq. (24), the SR model has also predicted the analytical formulation with high precision. The mathematical expression that the SR model has found for $M = 10$ is identified as:

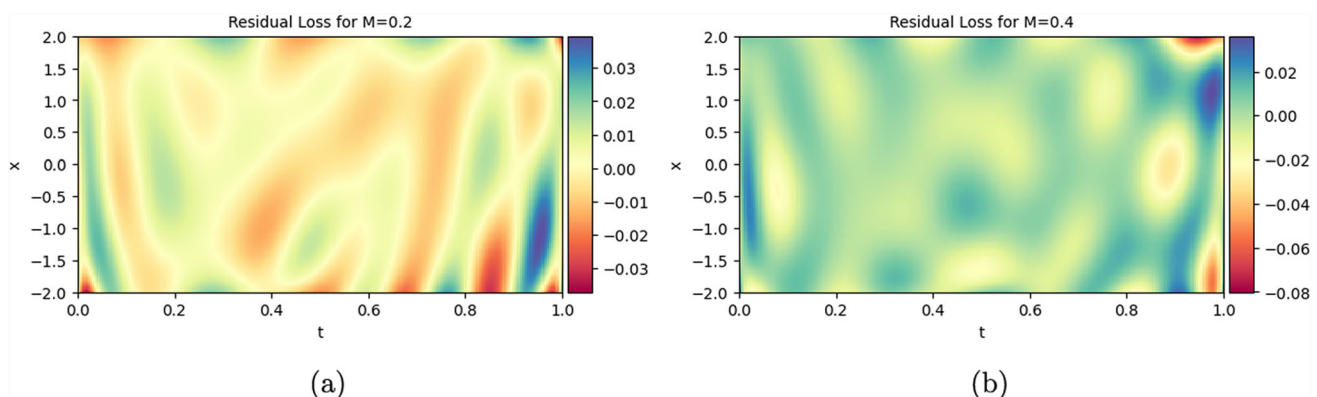


Fig. 15 Demonstrating the residual losses of fifth-order evolutionary PDE in a rectangular domain with varying parameter of M

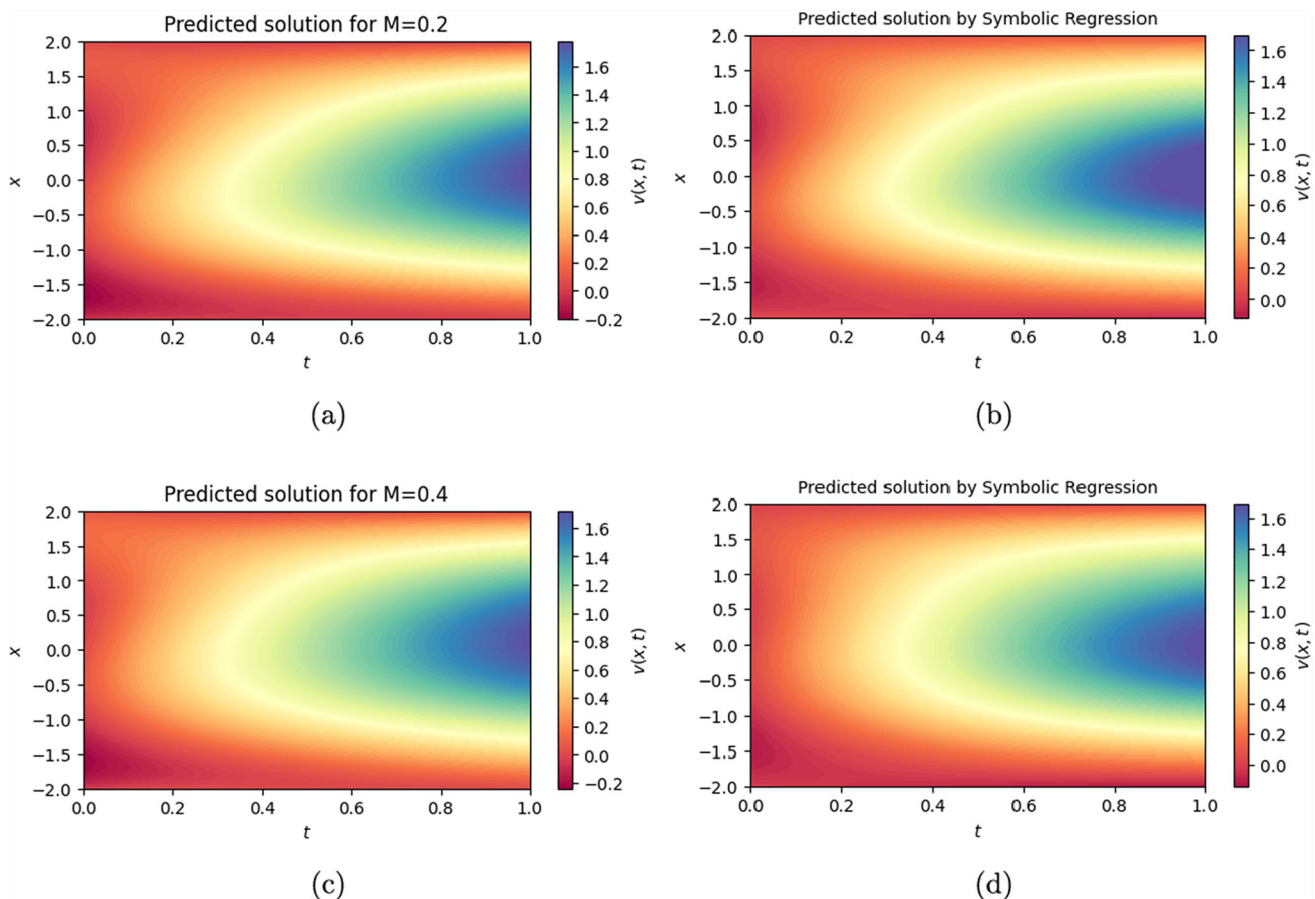


Fig. 16 Visualization of PINN-predicted solution and symbolic regression solution for fifth-order evolutionary equation with different M values

$$v(x, t) = \left(t \cdot 0.0619 - \cos\left(\frac{x}{0.318}\right) 0.166 - 0.919 \right) \sin\left(\frac{x}{0.318}\right) \quad (33)$$

For this equation, the mean absolute error between the PINN model prediction and the SR solution is 0.006. For Eq. (33), the MSE loss and complexity that we have achieved are 0.00005 and 17, respectively. This low MSE loss confirms that the discovered equation provides an excellent fit to the given data. A systematic summary of key performance indicators, including the mean residual loss, mean squared error, mean absolute error, and the complexity of the discovered equation, based on the predictions of the PINN model, is presented in Table 2. This table provides insight into how these metrics vary with different values of M , offering a comprehensive evaluation of the model's predictive accuracy and the interpretability of the resulting symbolic expressions.

Thus, the SR framework shows a strong capability in accurately capturing the underlying analytical form of Eq. (24), as demonstrated by the thorough analysis of the two-dimensional results and the associated MSE losses. This consistently high performance across a range of M values reflects the robustness and adaptability of the SR model in recovering physically meaningful expressions under varying dynamic conditions.

6.3 Symbolic regression solution for fifth-order evolutionary equation

Fifth-order partial differential equations are very challenging to solve due to the presence of nonlinearity and high-dimensional mathematical complexities. These PDEs are often hard to capture by using conventional numerical methods because these equations often contain complex boundary interactions within their respective

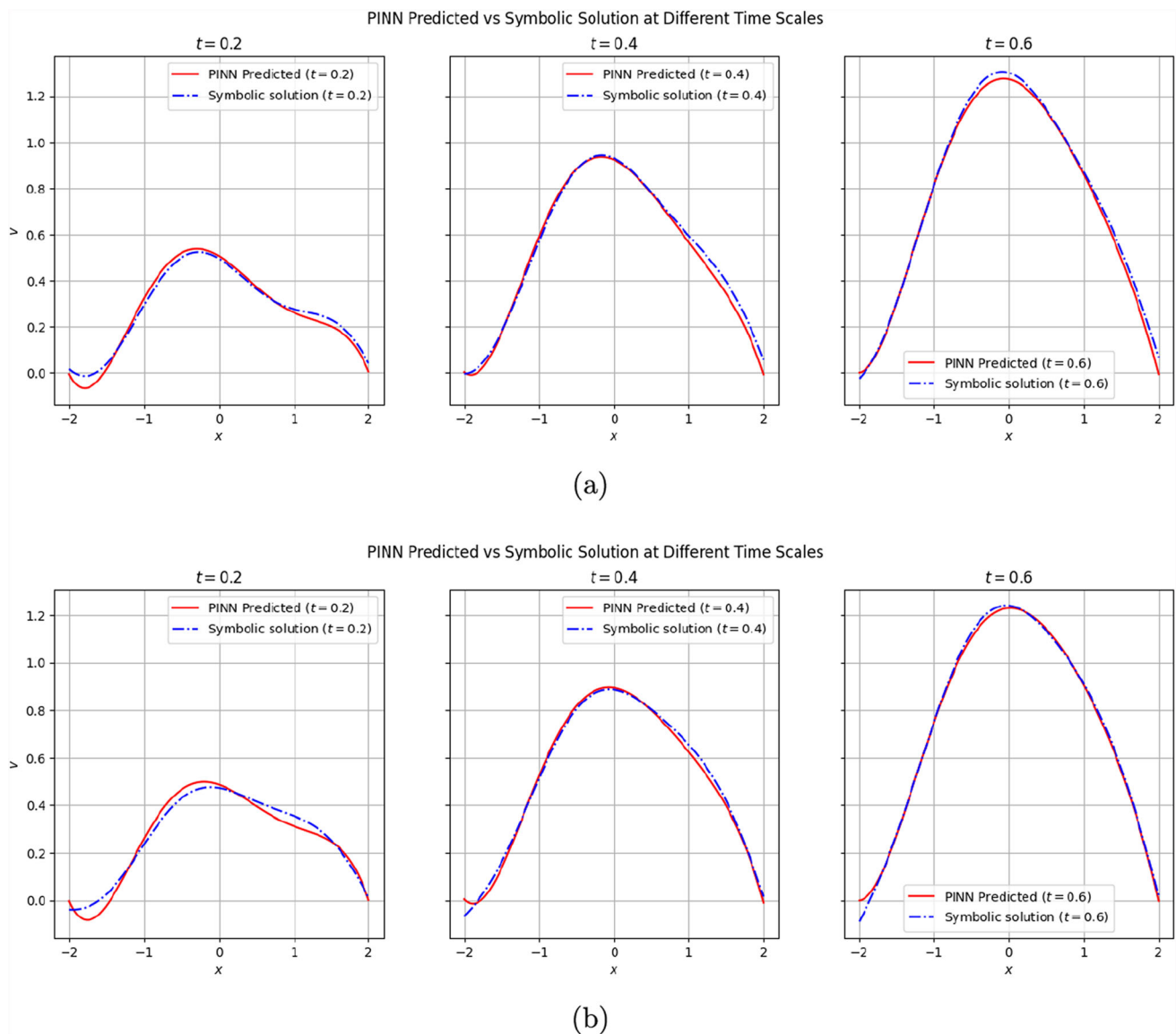


Fig. 17 A comparison of the fifth-order evolutionary equation estimates made using the PINN and symbolic regression approaches over a range of time scales with different values of M . Solutions for $M = 0.2$ and $M = 0.4$ are compared in (a) and (b), respectively

domains. In this context, PINNs have shown tremendous efficacy by making use of the fundamental physics that are included in the PDE. PINNs are capable of predicting highly accurate solutions in such scenarios where conventional numerical methods struggle. Maintaining physical consistency and generalization ability makes the PINNs a powerful tool to tackle such high-dimensional PDEs with nonlinear terms. In this case, the process of determining the solution to the PDEs was approached as a forward problem, which means that the governing equations and initial or boundary conditions are assumed to be known, and the goal was to compute the corresponding physical response of the system. The PINN architecture was set with 5 hidden layers and 30 neurons in each layer. For this instance, sine activation function was employed to update the network weights and biases during backpropagation in the PINN model, using a learning rate of 0.001. To solve Eq. (25), we have used two distinct small values of the parameter M and utilized the symbolic regression technique for individual predictions. The PINN model was trained for 20,000 for two different values of M . Figure 15a, b depicts the

residual loss distribution throughout the domain. It is visible from the figures that the residual losses have significantly lower values for two different parametric values of M . The mean residual loss for $M = 0.2$ became 0.006, and for the value of $M = 0.4$, the mean residual loss was 0.0007. These losses signify that our PINN models have achieved excellent results with great efficiency.

Following the training of our PINN model, the mathematical formulations of a fifth-order evolutionary equation with two different parameters of M were found using the SR model. For Eq. (25) with $M = 0.2$, the number of iterations was set to 34, whereas 30 was selected as the population size in the SR model with a batch size of 264. Similarly, for the value of $M = 0.4$ in Eq. (25), 40 iterations were selected with a population size of 30. Here, 201 evenly distributed points in x and t domains were utilized to find the symbolic regression solutions with the help of PINN-predicted values. Figure 16a–d presents a set of two-dimensional (2D) visualizations that highlight both the predictions that are generated by the PINN model and the outcomes of the symbolic regression analysis. These graphs are based on the projected solutions obtained from our neural network model and serve to illustrate how effectively the model captures the underlying dynamics of the system through both numerical approximation and interpretable symbolic expressions. A little but discernible difference between the outcomes of the PINN and symbolic regression may be seen in the two-dimensional graphs. This little variance raises the possibility that the symbolic regression model may have provided a more accurate or comprehensible depiction of the outcome in some areas of the domain by better capturing the underlying relationships within the provided information.

Figure 17a, b illustrates a detailed graphical comparison that showcases how effectively our SR model identifies precise and interpretable mathematical expressions that closely approximate the underlying dynamics governed by the fifth-order partial differential equation (i.e., Eq. 25). The figures reveal that the SR model captures the solution behavior with remarkable fidelity, demonstrating only minimal deviation from the actual model predictions. In particular, for the case where the magnetic parameter is set to $M = 0.2$, the analytical expression derived by the SR model exhibits a strong agreement with the reference data. This demonstrates the resilience of the symbolic regression approach and finds its capacity to provide physically significant equations that adhere to the system's governing physics. In this case, the SR model found the following resultant expression:

$$v(x, t) = (\cos(0.0656 - x) + 0.414) \cdot 1.70 \sin(\sin(0.0760)x + t) - \cos(0.177 - x)(x - 0.0541) \cdot 0.176 \quad (34)$$

For Eq. (34), the mean absolute error between the PINN predictions and the symbolic regression solution is impressively low, which is only 0.02. Again, the MSE loss associated with this equation is only 0.0008, which indicates a very tight fit between the PINN-predicted and symbolic expressions. The resulting symbolic expression has a complexity score of 27 that reflects a relatively concise yet expressive mathematical form. The combination of a low MAE and a small MSE loss demonstrates that the symbolic model attains both smooth convergence and high predictive accuracy, reflecting a well-fitted and physically consistent representation. This

Table 3 Summary of performance metrics for different M values in the fifth-order evolutionary equation, covering PINN residual loss, symbolic regression MSE, mean absolute error, and expression complexity

Loss metrics for fifth-order evolutionary equation		
Loss type	$M = 0.2$	$M = 0.4$
Mean residual loss	0.006	0.0007
MSE loss of symbolic regression	0.0008	0.0009
Mean absolute error between PINN and SR model	0.02	0.02
Complexity of discovered equation	27	23

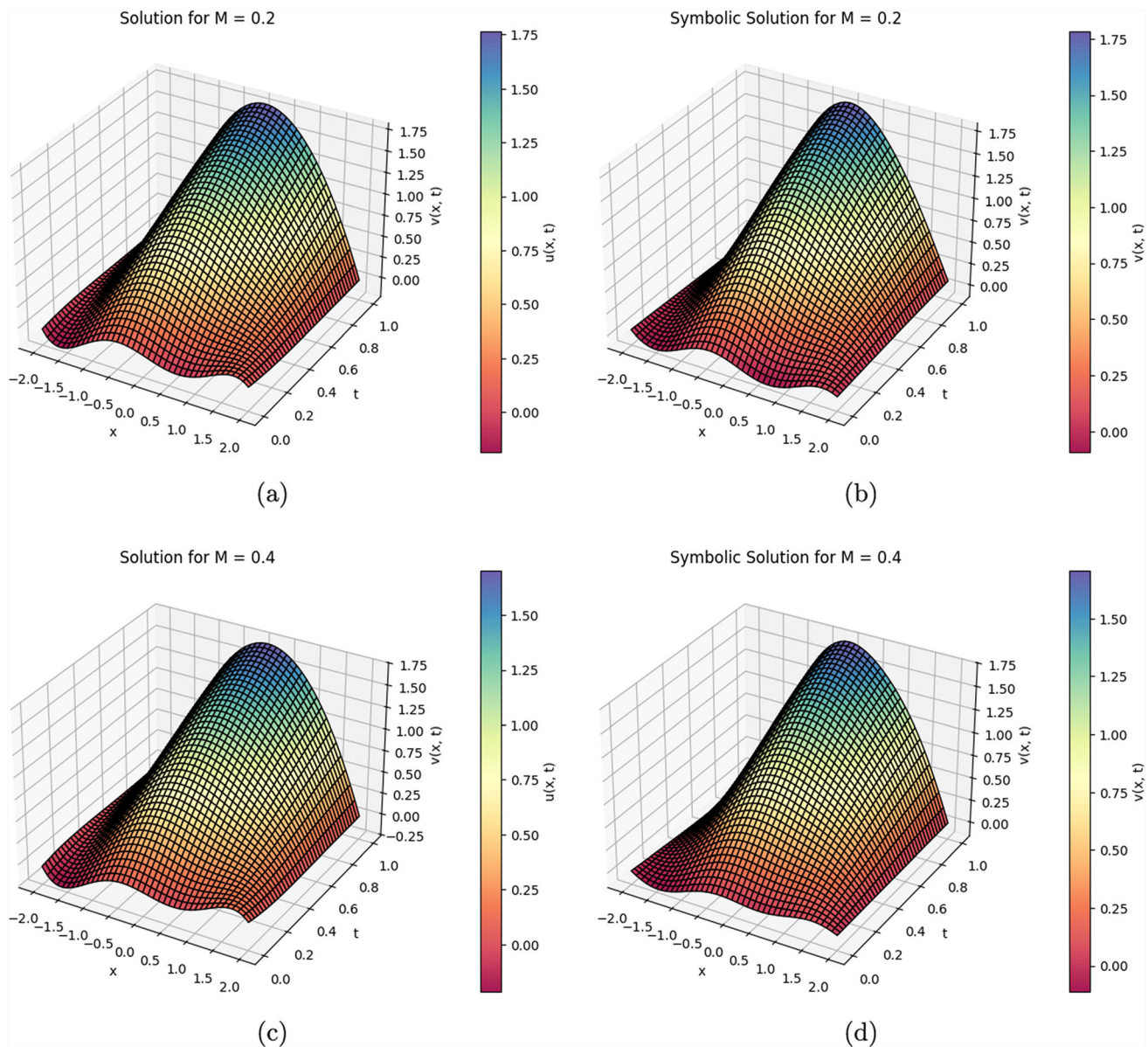


Fig. 18 3D plots of PINN-based predictions and SR solution, derived from obtained mathematical expressions, highlighting the model performances in achieving solution structures

further indicates that the model successfully captures the underlying dynamics encoded by the PINN. Again, for $M = 0.4$ in Eq. (25), the mathematical expression by the SR model is provided as:

$$v(x, t) = \cos(x \cdot 2.33)(x - 0.352)(-0.0687) + \cos(x \cdot 0.799 - 0.0383) \sin(\sin(t)) \cdot 2.27 \quad (35)$$

The accuracy of the SR model is evaluated by comparing its solution with the PINN prediction. The mean absolute error between the PINN solution and symbolic regression prediction for Eq. (35) is found to be 0.02, which is the same as for the equation with $M = 0.02$. To further evaluate how effectively the symbolic regression (SR) model captures the underlying relationship between the input variables and the predicted outputs, both the predictive accuracy and the structural complexity of the resulting expression were examined. The model achieved

Table 4 Comprehensive sensitivity analysis of the SR model over 27 evolutionary iterations to discover an accurate approximate equation for the solution of the forced Burgers' equation, evaluating how population size affects model accuracy, expression complexity, and runtime, and identifying balanced trade-offs between accuracy and simplicity

Symbolic regression—model sensitivity analysis				
Population size	MSE	Complexity	Runtime (s)	MAE
12	0.0021	24	13.60	0.03
18	0.00045	11	25.56	0.0107
22	0.00045	9	28.60	0.011
26	0.00007	29	38.03	0.005
30	0.00007	18	39.08	0.005
35	0.00042	24	41.64	0.011
40	0.00009	21	59.00	0.005
43	0.00008	18	70.00	0.006

a remarkably low MSE loss of just 0.0009, indicating a high level of agreement between the predicted and true values. Additionally, the complexity of the evolved expression was measured at 23, which is quantified by the total number of mathematical terms involved, reflecting a relatively compact yet expressive formulation of the learned relationship. This minimum MSE value, as well as Fig. 17, helps to understand the excellent performance in identifying the accurate and interpretable mathematical expressions with a given dataset in approximating the PDE solutions using the symbolic regression machine learning algorithm. Table 3 presents a detailed evaluation of losses of the PINN model's performance in solving the fifth-order evolutionary equation across different values of M . The table includes major indicators such as the mean residual loss, mean squared error from symbolic regression, the mean absolute error between the PINN and symbolic regression outputs, and the complexity of the derived symbolic expressions. Together, these metrics offer a comprehensive understanding of both the predictive accuracy of the model and the interpretability of the equations it discovers. Consequently, this approach yields accurate and suitable outcomes with very minimal errors. Some 3D plots of analytical solutions that have been derived from the SR model with input data (i.e., x and t) are visualized in Fig. 18a–d. These plots demonstrate the well-fitted structure with PINN solution data in its domain.

The combined physics-informed neural network and symbolic regression (PINN-SR) framework offers a powerful approach for deriving interpretable mathematical expressions from complex data. However, this enhanced capability comes with increased computational demands. Training the PINN requires substantial time and computational resources, especially for complex nonlinear higher-order PDEs on large datasets. Additionally, the symbolic regression component examines several mathematical expressions to identify the best fit with PINN predictive data, which can be very challenging and time-consuming. Strategies such as data preprocessing, early

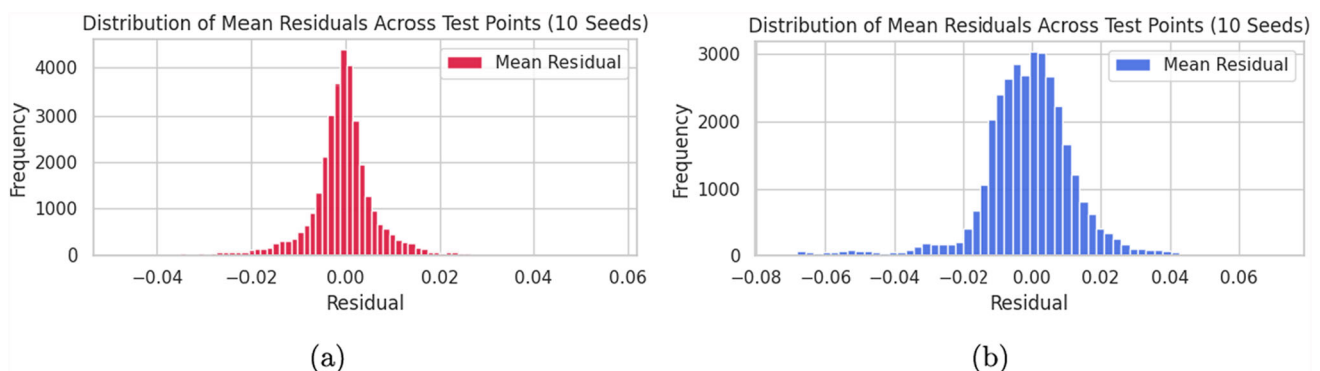


Fig. 19 Histogram of residual losses on test dataset for different seed values for creating diverse training and testing datasets

stopping, and parallel computing can help mitigate the computational costs and improve accuracy. A more detailed discussion of several challenges and recommendations is provided in Sect. 8.

6.4 Sensitivity analysis of hyperparameters in symbolic regression model

Sensitivity analysis is a critical component in the evaluation of machine learning and SR models. It provides insights into how variations in the hyperparameters of an SR model influence its overall performance. By systematically modifying input model parameters, such as population size in evolutionary algorithms, it is feasible to test the stability, accuracy, and interpretability of the resulting model. This procedure not only depicts optimal configurations for model training but also demonstrates the durability of the approach under varying computational constraints. In the framework of symbolic regression, where both prediction accuracy and expression simplicity are essential, sensitivity analysis is fundamental in guiding the choice of hyperparameters that provide effective and interpretable solutions. The effect of population size in the SR model has a highly impactful effect on finding better mathematical expressions with the predicted solution of the PINN model.

In our work, Table 4 presents the outcomes obtained by varying population sizes in the SR model with a fixed number of iterations. The SR model was executed once for each population size, as it exhibits sensitivity to input data and tends to produce varying outcomes across multiple runs. The results highlight how these changes impact expression complexity and error metrics such as MSE and MAE, reflecting the model's ability to fit the predicted data with accurate and interpretable mathematical expressions for the solution of the forced Burgers' equation with $M = 0.2$.

It is noticeable from Table 4 that increasing values of population size take more runtime of the model and slow down its convergence speed. Furthermore, the small population size accelerates its convergence, but it does not guarantee that it discovers the optimal mathematical equation that best fits the data of the predicted solution with the least complexity and minimum errors. Therefore, a suitable population size needs to be set to obtain optimal mathematical expressions with a fast convergence speed. High equation complexity can lead to overfitting and make the model difficult to interpret, and the discovered equation will be more challenging to understand analytically. Based on this analysis, 30 population size with 27 iterations in the SR model was selected to achieve the desired analytical expressions of the predicted data. As observed from Table 4, the SR model performs well with a population size of 30 and 27 iterations, achieving an MSE of 0.00007, an expression complexity of 18, and a mean absolute error of 0.005 when compared to the PINN model. Moreover, for larger population sizes, the SR model tended to produce highly complex mathematical expressions, which were significantly more difficult to interpret in a physical context compared to simpler expressions that exhibited minimal errors.



Fig. 20 Histogram of residual losses on unseen dataset for different seed values to make different datasets to capture the PINN model performances

7 Statistical and numerical validation of PINN predictions and computing confidence interval

7.1 Validation on test datasets

To evaluate the robustness and generalization capability of our PINN model, we trained it across 10 different random seeds that are used to generate new input datasets for training, and computed the mean PDE residuals with fixed $M = 0.2$ on a fixed unseen test set of 1500 points for each case.

For the forced Burgers' equation, the histogram (Fig. 19a) of residuals shows a tightly centered and approximately symmetric distribution around zero, indicating stable performance across different initializations. The mean residual across all seeds is found to be -4.91×10^{-4} , with a standard deviation of 5.03×10^{-3} and a 95% confidence interval of $\pm 3.11 \times 10^{-3}$. This suggests a high consistency and minimal bias in the data.

Similarly, for the fifth-order evolutionary equation, the histogram of PDE residuals is shown in Fig. 19b. The residuals are consistently low across different seeds, with a mean residual of -1.20×10^{-3} , standard deviation of 9.70×10^{-3} , and a 95% confidence interval of $\pm 6.01 \times 10^{-3}$. These results demonstrate that the model's performance is not sensitive to seed selection, affirming its reproducibility and generalization capability across different training scenarios.

Overall, the consistently low residuals and tight confidence intervals across different seed values confirm the reproducibility and generalization capability of the PINN models for both low-order and high-order nonlinear PDEs.

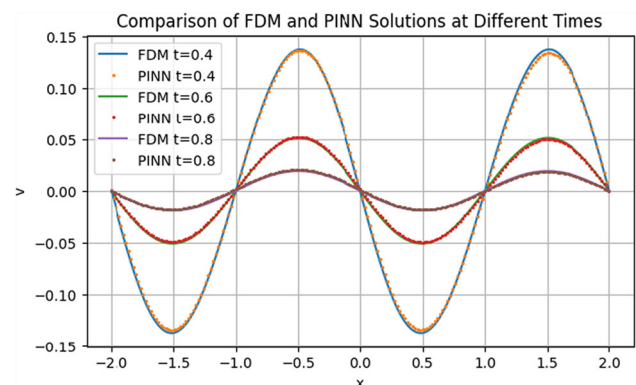
7.2 Cross-validation on unseen datasets of PINN predictions

To assess the robustness and generalization of our PINN models, we conducted a 10-fold cross-validation on two representative partial differential equations, denoted by Eqs. (24) and (25). For each equation, the PINN models were trained using 10 different random seed values to introduce diversity in the training points, and their performance was evaluated on a fixed, unseen test set with 20,000 data points in each fold.

In the case of Eq. (24), the histogram in Fig. 20a shows that the residual distribution is tightly centered around zero and displays a symmetric shape, indicating stable and repeatable convergence across folds. The average residual across all folds is approximately -3.64×10^{-4} , with a standard deviation of 7.16×10^{-3} and a 95% confidence interval of $\pm 1.14 \times 10^{-4}$. These results suggest that the PINN maintains both accuracy and stability across different training conditions.

For Eq. (25), the cross-validation results reveal almost similar pattern as shown in Fig. 20b. The residual distribution remains centered and symmetric, despite the equation's higher-order complexity. The mean residual

Fig. 21 Comparison of FDM and PINN solutions of the forced-Burgers equation with $M = 0$ at different time instances (i.e., $t = 0.4, 0.6, 0.8$). The close agreement between the two methods across the spatial domain demonstrates the accuracy of the PINN approach in capturing the solution dynamics



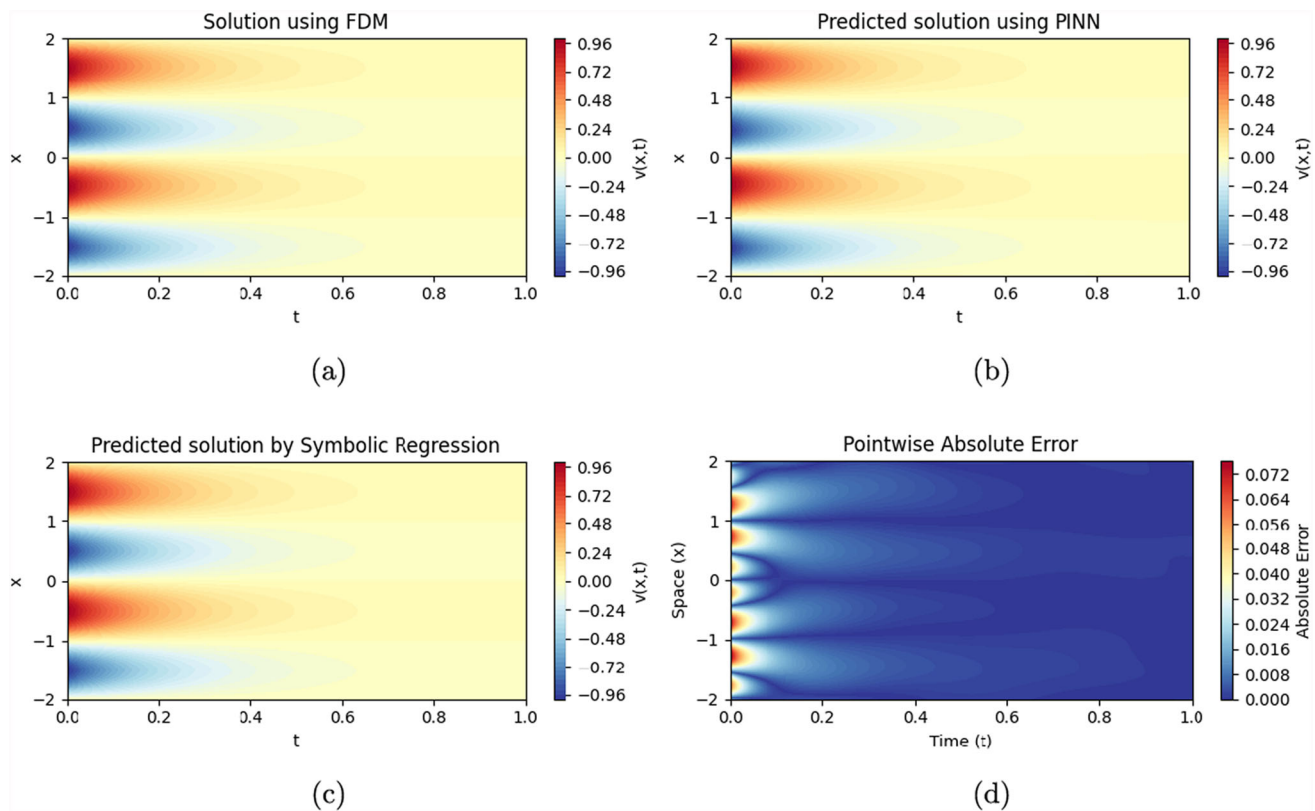


Fig. 22 Comparison of solution profiles and error analysis for the forced Burgers' equation with $M = 0$, demonstrating the validity of different computational approaches. **a** Numerical solution obtained using the FDM, serving as the reference. **b** Predicted solution from the PINN, showing close agreement with the FDM results across the spatial domain. **c** SR solution derived from the PINN-predicted data, highlighting the model's ability to extract interpretable and physically meaningful expressions. **d** Absolute error distribution between the PINN and FDM solutions over the spatio-temporal domain

is -2.55×10^{-4} , with a standard deviation of 1.29×10^{-2} and a 95% confidence interval of $\pm 5.66 \times 10^{-5}$. This demonstrates the strong consistency of the model.

Overall, the low variability in residuals across folds and the overlapping confidence intervals confirm that our PINN framework generalizes well and produces reproducible results. The use of cross-validation further validates the robustness of our method across different random initializations, making it a reliable approach for solving diverse nonlinear PDEs.

7.3 Numerical validation of deep learning-based solutions

To demonstrate the reliability and accuracy of the proposed method, a numerical approach has been employed to validate the results obtained from the applied technique. Specifically, the finite difference method (FDM) has been used to verify the outcomes of both the PINN and the SR methods. The numerical validation of the PINN solution for Eq. (24) with $M = 0$ is presented through a direct comparison with FDM results at different time instances $t = 0.4, 0.6, 0.8$, as shown in Fig. 21. This figure demonstrates that the PINN-predicted solutions show excellent agreement with the corresponding FDM results across the entire spatial domain $x \in [-2, 2]$.

Additionally, the PINN-based solution of Eq. (24), along with the solution derived from the interpretable mathematical expression provided by the SR model, has been compared with numerical solutions using FDM, as illustrated in Fig. 22a–c, using a periodic pulse wave as the initial condition. Figure 22d depicts the absolute error distribution between the PINN-predicted and FDM solutions over the domain. It is evident from the

figure that the errors remain sufficiently close to zero in most regions. These results provide strong evidence that the predicted solutions from the PINN align well with the numerical solutions, thereby confirming the accuracy and reliability of the proposed PINN approach. Thus, the application of deep learning methods in solving and analyzing nonlinear PDEs represents a promising alternative approach.

8 Some challenges and recommendations in symbolic regression

Symbolic regression is a powerful tool in modern computational algorithms, though it has some major issues while dealing with complex scientific problems. It finds better understandable mathematical expressions for some typical problems, but sometimes, the SR model discovers very complicated equations that may be too complex to interpret. To overcome the challenge of complicated equations in symbolic regression, it is beneficial to direct the SR model toward simpler solutions by imposing greater penalties on more complex mathematical operations. This approach helps to maintain a balance between accuracy and interpretability in the generated expressions. Keeping the overall complexity in check and prioritizing simpler formulations during model selection makes the outcomes clearer, more concise, and more accessible for scientific understanding. Furthermore, PySR encourages simpler solutions by applying parsimony pressure, which penalizes overly complex expressions through a tunable complexity weighting to prevent overfitting. This facilitates the identification of equations that are not only accurate but also interpretable and capable of generalizing to unseen data. PySR also uses multi-objective optimization to strike a balance between fitting the data well and keeping the equations simple. Techniques like early stopping are used to avoid overtraining, especially when performance stops improving on validation data. In addition, limiting the use of highly flexible operators helps reduce the chance of overfitting in noisy or limited datasets, making the results more robust and reliable.

Again, SR is computationally expensive for large, noisy datasets. The computational cost arises in identifying the predicted expressions. The population size and the number of iterations need to be rectified accordingly to enhance performance and reduce runtime, as these are the two main model parameters in symbolic regression. To enhance the efficiency of the SR model on large datasets, starting with data purification, noise removal, or dropping the number of input features might be beneficial. Using parallel computing or more advanced hardware may significantly decrease computational time. Additionally, incorporating early stopping when the model's performance levels off or testing smaller models before running the entire process can save time and resources. PySR tries to find closed-form analytical expressions by using the genetic algorithm on data. However, mathematical operators in predefined search spaces are very limited, as common functions generally include the sine function, cosine function, exponential function, and logarithmic function.

These mathematical functions with standard mathematical operators can be the cause of insufficiency for describing the governing physics of complex nonlinear PDEs due to the involvement of special functions such as Bessel and hypergeometric functions, and dependencies of non-polynomial form, like power law relationships in PDEs. To rectify this problem, a custom operator should be added to the PySR model so that it can effectively call this custom function during its training process for fitting the data with high performance speed and fewer errors. Furthermore, genetic programming algorithms may be stuck in local minima, failing to provide better or simpler equations that fit the given data. Mutation rates should be adjusted very carefully, or a large population size needs to be used to overcome this issue. Moreover, restarting the search process from a new starting point may be a useful way to avoid getting stuck in local minima. The method can explore other regions of the search space by beginning with a new batch of random solutions if the algorithm stops improving or converges to an undesirable result. This may increase the chances of finding better and more accurate results efficiently. Again, for very small datasets, the SR model may produce inaccurate predictions. It cannot read the perfect relationship among the data points for very few points in the datasets. To properly address the ideal, appropriate expressions to these data points, choosing sufficiently large data points inside the predetermined domain with proper tuning of model parameters is essential.

9 Scientific contributions and novelty

The primary scientific contribution of this work lies in the integration of physics-informed neural networks with symbolic regression to develop an interpretable, data-efficient framework for discovering approximate analytical solutions of nonlinear PDEs. Unlike traditional PINNs, which typically yield black-box solutions, the proposed hybrid approach facilitates the extraction of explicit analytical expressions that preserve physical meaning and enhance generalizability. This work stands out for its unique blend of deep learning and evolutionary symbolic regression, which aimed to find analytical solutions to nonlinear partial differential equations, specifically in the context of magnetically influenced biofluid dynamics. This remains a relatively unexplored area in current scientific research. Compared to prevailing techniques, which often focus solely on forward simulation or parameter estimation, our framework provides a transparent and compact representation of the system's behavior and has been validated through extensive numerical simulations and statistical analysis. This dual benefit of interpretability and predictive accuracy distinguishes the proposed method from existing approaches.

10 Physiological relevance and future directions

Bridging the gap between mathematical modeling and practical applications is very important, especially in fields like biology and medicine, where understanding the underlying physiological processes is crucial. This study shows that symbolic regression combined with PINN predictions can generate clear and concise mathematical expressions for nonlinear PDEs, such as the forced Burgers equation and a fifth-order evolutionary equation. These equations often reflect physiological phenomena such as nonlinear advection, viscous dissipation, and higher-order dispersive effects, which are central to understanding pressure wave propagation and flow dynamics in arteries. The solutions of these nonlinear PDEs offer insights into key physiological processes such as pressure wave propagation, arterial wall elasticity, and blood flow dynamics. This understanding is crucial for analyzing vascular conditions and can support the design of medical devices that respond to the natural behavior of the cardiovascular system. Increased arterial stiffness is closely linked to cardiovascular risks such as atherosclerosis, hypertension, and heart failure. Large artery stiffness is measured using pulse wave velocity because stiff arteries allow pulse waves to flow through them more quickly. This phenomenon can be used in clinical practice to predict cardiovascular events like heart attacks or strokes, since stiffer arteries make the heart work harder [43]. Additionally, surgical techniques such as angioplasty, along with pulse wave velocity monitoring, help evaluate recovery and assess the efficacy of the intervention in restoring normal vascular function.

Our main focus has been on mathematical accuracy and computational efficiency in obtaining the solutions of those PDEs. Still, investigating how these mathematical expressions relate to real physiological processes is an important goal for future work. Comparing the discovered expressions with established physiological models or experimental data could provide valuable insights into how well these equations reflect true biological mechanisms. In addition, collaboration with experts in medicine, physiology, and related fields is essential for accurately interpreting the results and assessing whether the derived equations provide meaningful insights or reliable predictions for biomedical applications. This collaborative and interdisciplinary effort will help turn mathematical findings into practical tools for guiding experiments, clinical decisions, and further scientific research.

11 Conclusions and remarks

In this study, the dynamics of nonlinear pulse propagation were investigated through the viscoelastic tube, specifically focusing on the deformation of the tube wall bending. A one-dimensional viscoelastic fluid flow model was utilized to identify the forced Burgers' equation and a fifth-order evolutionary equation that simulates the fluid flow nature in arteries. The main aspect of this research is to find the predicted solutions and their

analytical approximate expressions for the solutions of these nonlinear equations. In this regard, genetic programming, an evolutionary algorithm in machine learning approach, was used to successfully derive the mathematical expressions with high accuracy for the complex nonlinear systems. The following is a summary of the primary conclusions and contributions:

- The PINN algorithm with the symbolic regression technique was utilized to find the predicted solution as well as analytical expressions of forced Burgers' equation and nonlinear fifth-order evolutionary equation effectively with error $\mathcal{O}(10^{-3})$.
- A sensitivity analysis of the SR model's hyperparameters was conducted, demonstrating their impact on the model's performance and its ability to achieve accurate results.
- Solutions of the PDEs at different scales were highlighted, demonstrating the accuracy of the PINN model in satisfying the boundary conditions within the domain.
- The solutions of the PINN model and the symbolic regression model were compared, highlighting the superior performance of symbolic regression in producing well-fitted mathematical expressions for the PINN model's predicted data.
- The PINN model integrated with the symbolic regression method produces high-fidelity results, as demonstrated by both numerical and statistical validation. Residual loss analysis further supports the accuracy and reliability of the interpretable symbolic expressions.
- The MSE loss of the analytical solutions obtained through symbolic regression demonstrated a good fit and offered more profound insights into the relationship between input and output data, especially with appropriate population size adjustment and total number of iterations in the SR model to obtain optimum results.

Therefore, the mathematical expressions we obtained helped characterize the behavioral patterns of the PDEs and provided a better understanding of the nature of their solutions. Moreover, blood flow propagation, as well as its pulsatile nature in a viscoelastic artery, can be understood more accurately with the help of these discovered mathematical equations.

Overall, this research adds to the expanding understanding of how machine learning, especially PINNs, can be utilized to address nonlinear PDEs. The techniques described here not only increase simulation accuracy in intricate physical systems but also open the door for the PINN applications in symbolic regression to identify analytical solutions for scientific modeling in the future.

Acknowledgements The authors thank the reviewers and the associate editor for their comments and suggestions to improve the paper considerably. The author, J. Das, wishes to thank the Council of Scientific and Industrial Research (CSIR), India, for providing financial support (File No: 09/0028(21164)/2025-EMR-I) as a research scholar at the University of Calcutta, India.

Author Contributions JD involved in conceptualization (equal); data curation (equal); formal analysis (equal); investigation (equal); methodology (equal); software (equal); validation (equal); writing—original draft (equal). BB took part in conceptualization (equal); formal analysis (equal); writing—review and editing (equal). SD involved in conceptualization (equal); formal analysis (equal); funding acquisition (equal); supervision (equal); writing—review and editing (equal). SC involved in conceptualization (equal); formal analysis (equal); supervision (equal); writing—review and editing (equal).

Funding Open access funding provided by Copenhagen University. The financial support was provided by the Council of Scientific and Industrial Research (CSIR), India.

Data Availability The data that support the findings of this study are openly available in GitHub at <https://github.com/satyasaran/SymPINNPDE.git>.

Declarations

Conflict of interest The authors have no conflicts to disclose.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the

original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Thomas B, Sumam K (2016) Blood flow in human arterial system-a review. *Procedia Technol* 24:339–346
2. Ku DN (1997) Blood flow in arteries. *Annu Rev Fluid Mech* 29(1):399–434
3. Shah NA, Vieru D, Fetecau C (2016) Effects of the fractional order and magnetic field on the blood flow in cylindrical domains. *J Magn Magn Mater* 409:10–19
4. Lund LA, Ghoto AA, Al-Khaled K, Ghachem K, Fadhel MA, Khan SU, Kolsi L (2024) Thin film flow of blood-based hybrid nanoparticles subject to slip effects: a stability assessment. *Int J Mod Phys B* 38(15):2450183
5. Bhaumik B, Changdar S, Chakraverty S, De S (2024) Effects of viscosity and induced magnetic fields on weakly nonlinear wave transmission in a viscoelastic tube using physics-informed neural networks. *Phys Fluids* 36(12):121902
6. Wang X-F, Nishi S, Matsukawa M, Ghigo A, Lagr  e P-Y, Fullana J-M (2016) Fluid friction and wall viscosity of the 1d blood flow model. *J Biomech* 49(4):565–571
7. Kissas G, Yang Y, Hwuang E, Witschey WR, Detre JA, Perdikaris P (2020) Machine learning in cardiovascular flows modeling: predicting arterial blood pressure from non-invasive 4d flow mri data using physics-informed neural networks. *Comput Methods Appl Mech Eng* 358:112623
8. Samaniego E, Anitescu C, Goswami S, Nguyen-Thanh VM, Guo H, Hamdia K, Zhuang X, Rabczuk T (2020) An energy approach to the solution of partial differential equations in computational mechanics via machine learning: concepts, implementation and applications. *Comput Methods Appl Mech Eng* 362:112790
9. Mijwel MM, Esen A, Shamil A (2023) Overview of neural networks. *Babylonian J Mach Learn* 2023:42–45
10. Pratama A, Danang MA, Bakar IN (2022) Ann-based methods for solving partial differential equations: a survey. *Arab J Basic Appl Sci* 29(1):233–248
11. Raissi M, Perdikaris P, Karniadakis GE (2017) Physics informed deep learning (part i): data-driven solutions of nonlinear partial differential equations. [arXiv:1711.10561](https://arxiv.org/abs/1711.10561)
12. Li X, Liu Y, Liu Z (2023) Physics-informed neural network based on a new adaptive gradient descent algorithm for solving partial differential equations of flow problems. *Phys Fluids* 35(6):063608
13. Fuhg JN, Bouklas N (2022) The mixed deep energy method for resolving concentration features in finite strain hyperelasticity. *J Comput Phys* 451:110839
14. Shihab MA, Taha WM, Hameed RA, Jameel A, Ibrahim S (2023) Implementation of variational iteration method for various types of linear and nonlinear partial differential equations. *Int J Electr Comput Eng* 13(2):2131–2141
15. Altaie SA, Anakira N, Jameel A, Ababneh O, Qazza A, Alomari AK (2022) Homotopy analysis method analytical scheme for developing a solution to partial differential equations in fuzzy environment. *Fractal Fraction* 6(8):419
16. Shi P, Zeng Z, Liang T (2024) Physics-informed convnet: learning physical field from a shallow neural network. *Commun Nonlinear Sci Numer Simul* 132:107911
17. Xiang Z, Peng W, Zhou W, Yao W (2022) Hybrid finite difference with the physics-informed neural network for solving pde in complex geometries. [arXiv:2202.07926](https://arxiv.org/abs/2202.07926)
18. Majumdar R, Jadhav V, Deodhar A, Karande S, Vig L, Runkana V (2023) Symbolic regression for pdes using pruned differentiable programs. [arXiv:2303.07009](https://arxiv.org/abs/2303.07009)
19. Zhu Z, Hao J, Huang J, Huang B (2023) Bc-pinn: an adaptive physics informed neural network based on biased multiobjective coevolutionary algorithm. *Neural Comput Appl* 35(28):21093–21113
20. Oh H, Amici R, Bomarito G, Zhe S, Kirby R, Hochhalter J (2023) Genetic programming based symbolic regression for analytical solutions to differential equations. [arXiv:2302.03175](https://arxiv.org/abs/2302.03175)
21. Changdar S, Bhaumik B, Sadhukhan N, Pandey S, Mukhopadhyay S, De S, Bakalis S (2024) Integrating symbolic regression with physics-informed neural networks for simulating nonlinear wave dynamics in arterial blood flow. *Phys Fluids* 36(12):121924
22. Abbasi A, Al-Khaled K, Zouidi F, Khan SU, Khan MI, Bafakeeh OT, Farooq W, Choudhari R (2023) Blood-based electro-osmotic flow of non-newtonian nanofluid (carreau-yasuda) in a tapered channel with entropy generation. *ZAMM J Appl Math Mech Zeitschrift f  r Angewandte Mathematik und Mechanik* 103(5):e202100351
23. Wang X (2014) 1D modeling of blood flow in networks: numerical computing and applications. Ph.d thesis, Universit   Pierre et Marie Curie-Paris VI
24. Ahmad S, Al-Johani A, Sahu S (2021) Effect of magnetic and perturbation parameters on blood flow distribution through an artery. *Appl Appl Math Int J (AAM)* 16(1):32

25. Kudryashov N, Chernyavskii I (2008) Numerical simulation of the process of autoregulation of the arterial blood flow. *Fluid Dyn* 43(1):32–48
26. Nasrin R, Hossain A, Zahan I (2020) Blood flow analysis inside a stenotic artery using power-law fluid model. *RDMS* 13:1–10
27. Steele BN, Valdez-Jasso D, Haider MA, Olufsen MS (2011) Predicting arterial flow and pressure dynamics using a 1d fluid dynamics model with a viscoelastic wall. *SIAM J Appl Math* 71(4):1123–1143
28. Giusti A, Mainardi F (2016) A dynamic viscoelastic analogy for fluid-filled elastic tubes. *Meccanica* 51:2321–2330
29. Nikolova E, Jordanov I, Dimitrova Z, Vitanov N (1895) Evolution of nonlinear waves in a blood-filled artery with an aneurysm. *AIP Conf Proc* 1:2017
30. Kudryashov NA, Sinelshchikov DI (2011) Nonlinear evolution equation for describing waves in a viscoelastic tube. *Commun Nonlinear Sci Numer Simul* 16(6):2390–2396
31. Cuomo S, Di Cola VS, Giampaolo F, Rozza G, Raissi M, Piccialli F (2022) Scientific machine learning through physics-informed neural networks: Where we are and what's next. *J Sci Comput* 92(3):88
32. Faroughi SA, Pawar N, Fernandes C, Raissi M, Das S, Kalantari NK, Mahjour SK (2022) Physics-guided, physics-informed, and physics-encoded neural networks in scientific computing. [arXiv:2211.07377](https://arxiv.org/abs/2211.07377)
33. Lagaris IE, Likas A, Fotiadis DI (1997) Artificial neural network methods in quantum mechanics. *Comput Phys Commun* 104(1–3):1–14
34. Yu J, Lu L, Meng X, Karniadakis GE (2022) Gradient-enhanced physics-informed neural networks for forward and inverse pde problems. *Comput Methods Appl Mech Eng* 393:114823
35. Lu L, Meng X, Mao Z, Karniadakis GE (2021) Deepxde: a deep learning library for solving differential equations. *SIAM Rev* 63(1):208–228
36. Song Y, Wu Y, Guo Y, Yan R, Suganthan PN, Zhang Y, Pedrycz W, Das S, Mallipeddi R, Ajani OS et al (2024) Reinforcement learning-assisted evolutionary algorithm: a survey and research opportunities. *Swarm Evol Comput* 86:101517
37. Shukla A, Pandey HM, Mehrotra D (2015) Comparative review of selection techniques in genetic algorithm. *IEEE*, pp 515–519
38. Slowik A, Kwasnicka H (2020) Evolutionary algorithms and their applications to engineering problems. *Neural Comput Appl* 32:12363–12379
39. Kramer O, Kramer O (2017) *Genetic algorithms*. Springer, Berlin
40. Katoch S, Chauhan SS, Kumar V (2021) A review on genetic algorithm: past, present, and future. *Multimedia Tools Appl* 80:8091–8126
41. Cranmer M (2023) Interpretable machine learning for science with pysr and symbolicregression. [arXiv:2305.01582](https://arxiv.org/abs/2305.01582)
42. Tonda A (2025) Review of PySR: high-performance symbolic regression in python and julia. *Genet Program Evolvable Mach* 26:7
43. Inoue N, Maeda R, Kawakami H, Shokawa T, Yamamoto H, Ito C, Sasaki H (2009) Aortic pulse wave velocity predicts cardiovascular mortality in middle-aged and elderly Japanese men. *Circ J* 73(3):549–553

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Joy Das¹ · Bivas Bhaumik² · Soumen De¹ · Satyasan Changdar³

✉ Soumen De
sdeappmath@caluniv.ac.in

✉ Satyasan Changdar
sach@di.ku.dk; satyasan@gmail.com; satyasan@food.ku.dk

¹ Department of Applied Mathematics, University of Calcutta, 92, A.P.C. Road, Kolkata 700009, India

² Department of Mathematics, National Institute of Technology, Rourkela, Odisha, India

³ Department of Food Science, University of Copenhagen, Copenhagen, Denmark